Summer Associate Internship Exercises

Project Repository - github.com/sanaamironov/JPM

Executive Summary

This report implements and evaluates demand estimation approaches for credit-card merchant offers, where consumers choose among menus of offers and behavior can depend on the full context of the set. Part 1 focuses on **context-dependent discrete choice** via Zhang (2025)'s[1] DeepHalo model, implemented in a `choice-learn`-style API and validated with sanity checks targeting permutation equivariance and masking/availability behavior. Part 2 focuses on **unobserved market–product shocks and endogeneity** via Lu & Shimizu (2025)[2], including BLP-style benchmarks and a shrinkage estimator implemented using `tfp.mcmc` for posterior sampling / shrinkage inference.

- **Part 1 (Zhang 2025 / DeepHalo).** Implemented a DeepHalo estimator and reproduced context effects on synthetic menus. Added invariance/equivariance tests, masking checks, and optimization/gradient sanity checks to validate correct behavior under permutations and availability constraints.
- **Part 2 (Lu & Shimizu 2025 / sparse shocks).** Implemented the Section 4 simulation pipeline, including (i) BLP benchmark estimators (with and without CostIV variants) and (ii) a shrinkage estimator with inference implemented using `tfp.mcmc`. The goal is to recover demand parameters and sparse market–product shocks under endogeneity.
- **Hybrid extension (Question 3(d)).** Implemented a modified DeepHalo that incorporates Lu-style sparse unobserved shocks by adding a market–product shock term to utility and estimating it via MAP with an ℓ_1 sparsity penalty. A simulation study shows that the hybrid improves in-sample fit and qualitatively identifies a subset of injected sparse shocks (sensitivity 0.23–0.35, high specificity $\geq$0.99) relative to DeepHalo alone when the DGP contains sparse unobservables; support recovery is high-specificity but low-sensitivity at reported thresholds.
- **Bonus Question 1 (revised: storable goods, stockpiling, and strategic pricing).** Implemented a research prototype addressing the five requested components of the revised question: (i) a Bellman equation incorporating Halo effects, endogenous prices (by construction), integer inventory dynamics from Ching (2020), consumer-specific tastes η_{ij}, and consumer-specific discount factors, with explicit identifiability arguments per parameter; (ii) a two-stage MAP estimator implemented with TensorFlow and `tfp.distributions` (dynamic NLL plus a Bellman/TD penalty, spike-and-slab on d, Beta on π, Gaussian on μ and η), augmented by the Petrin & Train (2010) control function for price endogeneity using an observable cost shifter w_{jt} as the excluded instrument — all in graph mode without retracing; (iii) a simulation study with 95% Hessian-based local Laplace intervals (local curvature diagnostics; not validated frequentist coverage), showing that the control function roughly doubles the recovered price-sensitivity magnitude ($\hat{\beta}_{\text{price}}$ improves from -0.62 without CF to -1.09 with CF against true -1.50, recovering 73% of the true magnitude), with correct sign $\hat{\lambda}_{\text{control}} = +0.29$, and with $\hat{\mu}_t$ recovered in direction ($\text{corr}(\hat{\mu}, \mu^\star) = 0.62$) while η_{ij} is recovered at the noise floor (RMSE $\approx \sigma_\eta^\star$); the Laplace credible intervals are wide (SE $\approx$ 1.7 for $\hat{\beta}_{\text{price}}$), reflecting weak identification rather than precise calibration — see the Coverage Caveat in Section 19; (iv) a counterfactual price-promotion analysis showing how Halo and stockpiling mediate brand-X revenue response; and (v) a rigorous formulation of the joint price-and-assortment optimisation under a cardinality constraint, with a proposed nested greedy-plus-gradient algorithm.

Context dependence, unobserved shocks, and stockpiling dynamics address different failure modes: DeepHalo improves menu-aware prediction, sparse shocks target latent demand drivers and endogeneity, and a dynamic inventory state is needed for storable goods. The hybrid models combine these components and are validated via simulation-based recovery and smoke-test diagnostics.

Contents

Part I
Deep Context-Dependent Choice (Zhang 2025)

1 Introduction

Credit card issuers often run promotional campaigns in which cardholders see a menu of competing merchant offers, for example "spend $x, get $y back." The operational goal is not only to predict which offer is redeemed, but to estimate demand in a way that supports counterfactual planning: what happens to redemptions if we add or remove offers, change the incentive level, or shift campaign timing? Getting substitution patterns wrong leads to misallocation of marketing spend, because small changes to the menu can produce large changes in share. We model a decision instance by an individual i facing an offered set (menu) $\mathcal{C}_t$ at time t. Each alternative $j \in \mathcal{C}_t$ has observed features x_{ijt} (merchant category, discount depth, eligibility, time window) and latent appeal. Utility is written as

$$u_{ijt} = f_\theta(x_{ijt}, \mathcal{C}_t) + \epsilon_{ijt},$$

where f_θ is the systematic component and ϵ_{ijt} is an idiosyncratic shock. Classical discrete-choice baselines such as the multinomial logit (MNL) arise under i.i.d. extreme value shocks and a utility that depends only on x_{ijt}, not on the rest of the menu. This yields the Independence of Irrelevant Alternatives (IIA) property, implying that the odds between any two offers are invariant to the presence of other offers. Empirically, however, menu effects such as decoy, attraction, similarity, and compromise effects produce systematic IIA violations. In the offer setting, the perceived attractiveness of a discount can increase or decrease depending on which competing offers are simultaneously visible. Zhang et al. (2025)[1] propose the Deep Context-Dependent Choice Model (DeepHalo), which keeps a discrete-choice likelihood but replaces restrictive parametric utility forms with a permutation equivariant neural representation of the menu. The model is designed to learn context effects while respecting the set structure of $\mathcal{C}_t$. In this report we reconstruct DeepHalo in the `choice-learn` workflow using TensorFlow, validate the implementation on controlled synthetic experiments, and compare the approach to the Reproducing Kernel Hilbert Space (RKHS) choice model of Yang et al. (2025)[3]. The emphasis is on a transparent mapping between paper equations, implementation choices, and empirical behavior under menu perturbations.

What Part 1 delivers. Part 1 provides (i) a precise problem formulation with masking/availability notation, (ii) an implementation of DeepHalo compatible with `choice-learn`, (iii) verification checks (masking, permutation/indexing sanity checks, training stability), and (iv) synthetic demonstrations of canonical context effects.

2 Problem Formulation

We observe repeated discrete choices made by individuals over menus of competing offers. For each individual i and decision instance (time) t, the platform presents a finite choice set (menu) $\mathcal{C}_t \subseteq \{1, \dots, J\}$. Each offered alternative $j \in \mathcal{C}_t$ is associated with observed covariates $x_{ijt} \in \mathbb{R}^{d_x}$, and we observe the realized choice $y_{it} \in \mathcal{C}_t$. We represent the total utility of alternative j for individual i at time t as $U_{ijt} = u_{ijt} + \epsilon_{ijt}$, where u_{ijt} is the systematic component and ϵ_{ijt} is an idiosyncratic shock. The systematic utility is allowed to depend on both the alternative's covariates and the full menu, and we write it as

$$u_{ijt} = f_\theta(x_{ijt}, \mathcal{C}_t),$$

where f_θ is a parametric function indexed by parameters θ. For implementation and batching, it is convenient to encode each menu using an availability mask. For each instance t, we define $m_t \in \{0,1\}^J$ with $(m_t)_j = \mathbf{1}\{j \in \mathcal{C}_t\}$, so that $(m_t)_j$ indicates whether item j is available. In a batch of size B, we stack these masks to obtain $m \in \{0,1\}^{B \times J}$. This representation makes it explicit that the model must assign zero probability to unavailable items and renormalize over the offered set.

2.1 Baseline: Multinomial Logit and IIA

A standard baseline is the multinomial logit (MNL) model, in which the systematic utility depends only on the chosen item's features through a linear index,

$$u_{ijt} = \beta^\top x_{ijt},$$

and the shocks ϵ_{ijt} are i.i.d. Type-I extreme value. Under these assumptions, the choice probability for selecting alternative $j \in \mathcal{C}_t$ takes the softmax form

$$\mathbb{P}(y_{it} = j \mid \mathcal{C}_t) = \frac{\exp(\beta^\top x_{ijt})}{\sum_{k \in \mathcal{C}_t} \exp(\beta^\top x_{ikt})}.$$

A defining implication of MNL is the Independence of Irrelevant Alternatives (IIA) property. In particular, for any two alternatives $j, k \in \mathcal{C}_t$, the odds ratio is

$$\frac{\mathbb{P}(y_{it} = j \mid \mathcal{C}_t)}{\mathbb{P}(y_{it} = k \mid \mathcal{C}_t)} = \exp\left(\beta^\top (x_{ijt} - x_{ikt})\right),$$

which does not depend on what other alternatives are included in $\mathcal{C}_t$. In many marketing settings this invariance is too restrictive, because the perceived attractiveness of an offer can change when the menu changes due to comparison effects and other forms of context dependence.

2.2 Context-Dependent Discrete Choice

To allow menu effects, we retain the discrete-choice likelihood but generalize the systematic utility to depend on the full choice set. Specifically, we permit

$$u_{ijt} = f_\theta(x_{ijt}, \mathcal{C}_t),$$

while maintaining i.i.d. Type-I extreme value shocks so that choice probabilities remain a softmax over the offered set:

$$\mathbb{P}(y_{it} = j \mid \mathcal{C}_t; \theta) = \frac{\exp(u_{ijt})}{\sum_{k \in \mathcal{C}_t} \exp(u_{ikt})}.$$

Using the availability mask, the same probability can be written in a form that makes feasibility explicit:

$$\mathbb{P}(y_{it} = j \mid m_t; \theta) = \frac{(m_t)_j \exp(u_{ijt})}{\sum_{k=1}^{J} (m_t)_k \exp(u_{ikt})}.$$

This expression highlights that items with $(m_t)_j = 0$ must receive probability zero, and it provides a convenient implementation interface for variable size menus via masking.

2.3 Estimation Objective

Given data $\{(x_{ijt}, \mathcal{C}_t, y_{it})\}_{i,t}$, we estimate θ by maximum likelihood. Equivalently, we minimize the negative log-likelihood (NLL),

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i,t} \log \mathbb{P}(y_{it} \mid \mathcal{C}_t; \theta),$$

where N is the number of observed choices. For neural parameterizations of f_θ, we optimize $\mathcal{L}(\theta)$ using stochastic gradient methods. Because utilities under a softmax likelihood are identified only up to additive constants, empirical validation is most naturally conducted at the level of predicted probabilities and probability shifts under controlled menu perturbations rather than by interpreting absolute utility levels.

3 Literature Review

3.1 Deep Context Dependent Choice Model (Zhang, 2025)

3.1.1 Problem framing

Zhang et al. (2025)[1] address a central limitation of classical discrete-choice models, namely the Independence of Irrelevant Alternatives (IIA). Under IIA, relative substitution patterns between two alternatives are invariant to the presence of additional options. Empirically, however, context effects such as decoy, attraction, similarity, and compromise effects generate systematic IIA violations, which are especially relevant when consumers evaluate offers comparatively within a menu. The authors seek a model that incorporates rich item level features, represents higher order context-dependent interactions beyond simple pairwise adjustments, and remains permutation equivariant with respect to the offered set.

3.1.2 Utility decomposition

For a finite offered set S and an alternative $j \in S$, Zhang et al. motivate context dependence through an interaction order decomposition of the menu dependent utility:

$$u_j(S) = v_j^{(0)} + \sum_{k \in S \setminus \{j\}} v_j^{(1)}(\{k\}) + \sum_{\{k,\ell\} \subset S \setminus \{j\}} v_j^{(2)}(\{k,\ell\}) + \cdots, \tag{1}$$

where $v_j^{(0)}$ is the context-independent base utility, $v_j^{(1)}$ captures pairwise interactions between j and other items in the menu, $v_j^{(2)}$ captures second order interactions, and higher order terms capture interactions among larger subsets. The representation is intended to be compatible with permutation equivariance, meaning that relabeling or reordering alternatives does not change the implied choice behavior. Permutation equivariance can be stated as follows: for any permutation π acting on the item labels in S, the model satisfies

$$u_{\pi(j)}(\pi(S)) = u_j(S),$$

which ensures that utilities (and therefore probabilities) are invariant to ordering artifacts.

3.1.3 Neural parameterization

Zhang et al. implement the decomposition in Eq. (1) using stacked residual "halo blocks" that combine (i) a permutation invariant aggregation over the set and (ii) an item wise residual update. Each feature vector $x_j \in \mathbb{R}^d$ is first mapped to an embedding

$$z_j^{(0)} = \chi(x_j),$$

where χ is a shared multilayer perceptron. At layer ℓ, the model forms a context summary via a symmetric set function, and updates each item embedding with head specific interaction maps. A convenient high level description is

$$c^{(\ell)} \;=\; \mathrm{Agg}^{(\ell)}\Big(\{z_k^{(\ell-1)} : k \in S\}\Big), \qquad z_j^{(\ell)} \;=\; z_j^{(\ell-1)} + \frac{1}{H}\sum_{h=1}^{H} \phi_h^{(\ell)}\Big(z_j^{(\ell-1)},\, c^{(\ell)}\Big),$$

where $\mathrm{Agg}^{(\ell)}(\cdot)$ is permutation invariant and the mapping $S \mapsto \{z_j^{(\ell)}\}_{j \in S}$ is permutation equivariant. In our TensorFlow implementation, we instantiate this template with a masked mean context summary and head specific MLPs applied to concatenated vectors. Concretely, we compute $c^{(\ell)}$ using a masked mean over available items and define

$$\Delta z_j^{(\ell)} = \frac{1}{H}\sum_{h=1}^{H} \mathrm{MLP}_h^{(\ell)}\Big([z_j^{(\ell-1)} \| c^{(\ell)}]\Big), \qquad z_j^{(\ell)} = \mathrm{LayerNorm}\Big(z_j^{(\ell-1)} + \Delta z_j^{(\ell)}\Big).$$

This choice preserves the invariances emphasized by Zhang et al. (permutation equivariance and correct handling of masked availability) and matches the behavior validated by our synthetic context-effect experiments.

3.1.4 Choice probabilities, approximation, and identifiability

Utilities are taken to be linear in the final embedding,

$$V_j(S) = \beta^\top z_j^{(L)},$$

and choice probabilities follow the standard softmax likelihood,

$$P(j \mid S) = \frac{\exp(V_j(S))}{\sum_{k \in S} \exp(V_k(S))}.$$

In the featureless case (e.g., one-hot item encodings), the recursive structure can be interpreted as generating increasingly rich interaction terms over set indicators as depth increases. Zhang et al. provide a universal approximation argument over finite sets, although the proof does not directly yield minimal depth requirements or parameter growth rates. Identifiability is limited by the translation invariance of the softmax: adding any set specific constant $c(S)$ to all $V_j(S)$ leaves probabilities unchanged, so utilities are identified only up to additive constants that can vary with the menu.

3.1.5 Estimation

Parameters are estimated by maximum likelihood,

$$\mathcal{L}(\theta) = -\sum_{i,t} \log P(y_{it} \mid S_t; \theta),$$

and optimized using stochastic gradient methods such as Adam. The resulting objective is non-convex in θ, so optimization guarantees are local and practical performance depends on architecture, regularization, and implementation details that preserve the required invariances.

3.2 RKHS Choice Model (Yang, 2025)

3.2.1 Problem framing

Yang et al. (2025)[3] propose a function space alternative to deep permutation equivariant networks. Instead of parameterizing utilities with a neural architecture, the RKHS approach places menu dependent utility functions in a vector valued Reproducing Kernel Hilbert Space, which yields explicit regularization through the RKHS norm and can lead to convex objectives once the kernel is fixed.

3.2.2 Menu dependent utility

For a menu $S \subseteq \{1, \ldots, J\}$ and an alternative $j \in S$, Yang models utility using the RKHS inner product

$$u_j(S) = \langle u,\ K(\cdot, S) e_j \rangle_{\mathcal{H}}, \qquad u \in \mathcal{H},$$

where K is a positive definite matrix valued kernel and e_j is the basis vector selecting coordinate j. Choice probabilities again follow a softmax over the offered set,

$$p_j(S) = \frac{\exp(u_j(S))}{\sum_{k \in S} \exp(u_k(S))}.$$

In this formulation, context dependence is induced by the kernel through its dependence on S, rather than by explicitly stacking interaction blocks.

3.2.3 Feature-based extension

When feature matrices X_S are observed, kernels can be constructed to separate set structure from feature similarity through factorizations of the form

$$K\big((S, X_S), (S', X_{S'})\big) = K_S(S, S') \odot K_X(X_S, X_{S'}),$$

which allows the model designer to control menu interaction structure and attribute similarity independently. Polynomial set kernels of degree p can capture interactions up to order p, and the practical behavior of the estimator depends on these kernel choices.

3.2.4 Regularized estimation and computation

Estimation is typically performed through regularized empirical risk minimization in the RKHS,

$$\min_{u \in \mathcal{H}} \frac{1}{N} \sum_{i=1}^{N} \ell(y_i, p_i(u)) + \lambda \|u\|_{\mathcal{H}}^2,$$

where $\ell(\cdot)$ is the negative log-likelihood induced by the softmax probabilities. By the Representer Theorem, the solution can be expressed as a finite kernel expansion,

$$u(\cdot) = \sum_{i=1}^{N} K(\cdot, S_i)\alpha_i,$$

which reduces the problem to optimization over expansion coefficients. The primary computational challenge is that exact kernel computation and storage can scale poorly with the number of observations and alternatives, and Yang discusses approximations such as random features and related constructions to reduce memory and runtime.

3.3 Comparative analysis

DeepHalo and the RKHS choice model both relax IIA by allowing utilities to depend on the menu, but they impose different inductive biases and lead to different estimation trade offs. DeepHalo represents menu dependence through a parametric, permutation equivariant network trained with non-convex stochastic optimization, which can scale well with data and flexibly capture complex interactions, but whose behavior depends sensitively on architectural and regularization choices. The RKHS approach specifies a hypothesis space through a kernel and controls complexity explicitly via an RKHS norm, which can yield more transparent regularization and stronger theoretical footing once the kernel is fixed, but which may incur higher computational cost and requires careful kernel design to capture the relevant context effects. For credit card offer settings, the practical decision often comes down to whether the priority is expressive menu aware prediction under large scale data (favoring DeepHalo) or stability and explicit regularization with a well motivated kernel class (favoring RKHS), noting that neither approach by itself resolves endogeneity in offer assignment.

4 Model Setup and Interpretation Issues in Zhang (2025)

We implemented the Deep Context-Dependent Choice Model (*DeepHalo*) of Zhang et al. (2025)[1] as a TensorFlow model with a `choice-learn`-style interface. The codebase contains two execution paths selected by a single configuration flag, `authors_mode`. When `authors_mode=True`, the model uses a TensorFlow port of the authors' released architecture (our `AuthorsFeaturelessNetTF` / `AuthorsFeatureBasedNetTF`) and bypasses the simplified halo stack. When `authors_mode=False`, the model uses a separate, permutation-equivariant halo architecture (our `BaseEncoder` + `HaloBlock`) that we keep as an ablation and diagnostic baseline.

4.1 Ambiguities and Implementation Gaps

4.1.1 Notational inconsistencies in set functions

The paper introduces set functions such as $u_j(\cdot)$ and $v_j(\cdot)$ to formalize halo-style context dependence, but the notation alternates between object indexed forms such as $u_j(S)$ and function call forms such as $u(j, S)$. In addition, several expressions rely on the idea that context terms depend on the menu S and (conceptually) on subsets of $S \setminus \{j\}$, but the restriction "exclude the focal item" is sometimes left implicit. In our implementation we do not enumerate subsets; instead, we enforce the intended set dependence through explicit availability masking in the forward pass, so that unavailable items do not contribute to context aggregates and receive zero probability under the masked softmax. This is the concrete mechanism by which the implementation respects the paper's menu based semantics.

4.1.2 Multi-head aggregation rule not explicitly defined

The manuscript describes multi-head transformations in the halo block but does not always state how head outputs are combined before the residual update. This matters because concatenation, summation, and averaging imply different scaling as the number of heads changes. Our code makes this distinction explicit across the two execution paths: in the `authors_mode=True` path, head aggregation follows the structure implemented inside `AuthorsFeaturelessNetTF` / `AuthorsFeatureBasedNetTF` (a direct TensorFlow port of the authors style module). In the `authors_mode=False` ablation path, we aggregate heads by averaging,

$$\phi(z, c) = \frac{1}{H} \sum_{h=1}^{H} \phi_h(z, c),$$

which stabilizes the scale of updates as H varies and was empirically stable in our synthetic experiments.

4.1.3 Ordering of residual connections and normalization layers

The paper does not always specify whether normalization is applied before or after the residual addition, and pre-norm versus post-norm can change optimization behavior in deeper networks. In our simplified halo stack (`authors_mode=False`), we implement a post-normalization update,

$$z^{(\ell)} = \mathrm{LayerNorm}\Big(z^{(\ell-1)} + \phi(z^{(\ell-1)}, c)\Big),$$

which is standard for residual blocks and worked reliably in our experiments. In `authors_mode=True`, normalization and residual ordering follow the structure encoded in the authors-mode port modules.

4.1.4 Identifiability constraints are not fully specified

Although the paper discusses relative halo effects, it does not formally state the normalization constraints required for identifiability under the softmax likelihood. Because adding a set-specific constant $c(S)$ to all utilities in a menu leaves choice probabilities unchanged, utilities are identified only up to additive constants. The most robust quantities to validate are therefore choice probabilities and probability shifts induced by controlled menu perturbations, rather than absolute utility levels.

4.1.5 Depth interaction correspondence

The paper informally links network depth to interaction order, which is most literal under a featureless polynomial-expansion interpretation. In feature-rich settings, additional layers clearly increase representational capacity and enable richer menu-dependent transformations, but the resulting function class does not necessarily map cleanly to a finite order interaction expansion. For this reason, we interpret depth primarily as increasing functional capacity for context dependence while preserving permutation equivariant set structure.

4.1.6 Presentation issues in synthetic examples

Some presentation choices in the synthetic examples introduce minor replication ambiguity. Ordered tuples are occasionally used to denote sets, which can be misread as implying order sensitivity, and some figures use (i, j) notation that must be interpreted as "choosing i when j is present" rather than as an ordered pair. These issues are cosmetic rather than conceptual, but resolving them is necessary to reproduce the synthetic demonstrations faithfully.

5 DeepHalo Implementation in `choice-learn`

We implemented the Deep Context-Dependent Choice Model (*DeepHalo*) of Zhang et al. (2025)[1] as a TensorFlow model with a `choice-learn`-style interface. The implementation is organized around a single configuration flag, `authors_mode`, which selects between two execution paths that live in the same codebase. When `authors_mode=True`, the top-level `DeepHalo` module delegates its forward pass to `AuthorsFeaturelessNetTF` (featureless) or `AuthorsFeatureBasedNetTF` (feature-based), which are intended as a TensorFlow port of the authors' released architecture. When `authors_mode=False`, `DeepHalo` instead uses a separate permutation-equivariant halo stack (`BaseEncoder` + a stack of `HaloBlocks`) that we keep as an explicit ablation and diagnostic baseline.

5.1 Inputs, masking, and the training objective

Each training example is a choice set (menu) represented by an availability mask

$$m \in \{0,1\}^{B\times J},$$

where $m_{bj} = 1$ indicates item j is available in observation b. The model outputs utilities u_{bj} and masked log-probabilities $\log P(y_b = j \mid m_b)$ using a masked log-softmax, so that unavailable items receive probability zero:

$$\log P(y_b = j \mid m_b) \;=\; \log \frac{m_{bj}\exp(u_{bj})}{\sum_{k=1}^{J} m_{bk}\exp(u_{bk})}.$$

Training minimizes the mean negative log-likelihood,

$$\mathcal{L}(\theta) = -\frac{1}{B}\sum_{b=1}^{B} \log P(y_b \mid m_b;\theta),$$

and all of our correctness checks (permutation/masking tests and synthetic context-effect experiments) are evaluated at the probability level, since utilities are identified only up to set-specific additive constants under softmax.

5.2 One codebase, two implementations (`authors_mode`)

The routing logic in our top-level `DeepHalo` model is:

Authors-mode replication path (`authors_mode=True`). If `cfg.authors_mode=True`, `DeepHalo.call` bypasses the simplified halo stack entirely and returns the logits and masked log-probabilities produced by `AuthorsFeaturelessNetTF` (featureless) or `AuthorsFeatureBasedNetTF` (feature-based). This is the path used for replication-oriented experiments in this report.

Simplified / ablation path (`authors_mode=False`). If `cfg.authors_mode=False`, `DeepHalo` uses `BaseEncoder` to produce item embeddings and applies a stack of `HaloBlock` layers that implement a permutation-equivariant context update using masked mean pooling and multi-head MLP residual updates. This path is included as a transparent baseline and diagnostic variant; it is not claimed to match the internal block construction in the authors' released architecture.

5.3 Authors replication path (`authors_mode=True`)

5.3.1 Featureless case

In the featureless setting, each menu is encoded by an availability indicator vector $e_{S_b} \in \{0,1\}^J$ for observation b. The authors-mode featureless network produces logits

$$u_b \;=\; g_\theta(e_{S_b}) \in \mathbb{R}^J,$$

and the final masked log-probabilities are computed via the masked log-softmax shown above. In our implementation, this path is activated when `cfg.featureless=True` and `cfg.authors_mode=True`. The authors-mode code supports the block families exposed by the configuration option `authors_block_types` (e.g., exponential/multiplicative and quadratic-style residual blocks as described by the authors' model description), and it is the code path we use whenever we claim "authors-mode" results.

5.3.2 Feature-based case

In the feature-based setting, each menu provides a feature tensor $X_b \in \mathbb{R}^{J \times d_x}$ together with the availability mask m_b. The authors-mode feature-based network produces logits

$$u_b \;=\; g_\theta(X_b, m_b) \in \mathbb{R}^J,$$

followed by the same masked log-softmax. This path is activated when `cfg.featureless=False` and `cfg.authors_mode=True`.

5.4 Simplified / ablation path (`authors_mode=False`)

When `authors_mode=False`, our `DeepHalo` model uses a separate halo stack that is explicitly permutation-equivariant. This path is maintained as a baseline that preserves the key invariance properties and integrates cleanly with the rest of the `choice-learn`-style pipeline.

5.4.1 Base encoder

The `BaseEncoder` maps each item to an embedding in $\mathbb{R}^d$. In the featureless case, this is an embedding lookup from item IDs. In the feature-based case, this is an MLP projection of X_b. The base representation is

$$z^{(0)} \in \mathbb{R}^{B \times J \times d}.$$

5.4.2 HaloBlock update

Let $z^{(\ell-1)}$ denote the current representation and m the availability mask. Each `HaloBlock` computes a masked mean context vector

$$c_b \;=\; \frac{1}{\sum_{j=1}^{J} m_{bj}} \sum_{j=1}^{J} m_{bj}\, z_{bj}^{(\ell-1)} \in \mathbb{R}^d.$$

The block then forms an item-wise input by concatenating an item representation and the context summary. In our implementation, the residual variant controls whether the item representation is the current state or a fixed base:

$$\text{if standard:} \quad \tilde{z}_{bj} = z_{bj}^{(\ell-1)}, \qquad \text{if fixed\_base:} \quad \tilde{z}_{bj} = z_{bj}^{(0)}.$$

With H heads, each head is an MLP $\phi_h : \mathbb{R}^{2d} \to \mathbb{R}^d$, and the update is

$$\Delta z_{bj} \;=\; \frac{1}{H} \sum_{h=1}^{H} \phi_h([\tilde{z}_{bj} \,\|\, c_b]).$$

Finally, we apply a residual connection and layer normalization:

$$z_{bj}^{(\ell)} \;=\; \mathrm{LayerNorm}\Big(z_{bj}^{(\ell-1)} + \Delta z_{bj}\Big).$$

After L halo blocks, a linear layer maps the final embeddings to scalar utilities $u_{bj} = w^\top z_{bj}^{(L)}$ and masked log-probabilities are computed by masked log-softmax.

5.5 What we test and where outputs live in the repo

The test suite uses Python `unittest`. Part 1 tests cover masking behavior, padding invariance, input validation, and wrapper behavior (see `tests/part1/`). Part 2 tests and smoke runs are in `tests/part2/` and write artifacts under `results/`. The Zhang–Lu hybrid model in Part 2(d) writes experiment folders under `results/hybrid/zhang_lu_sparse/`. The Lu & Shimizu replication driver in Part 2(b) writes `summary.csv`, `paper_table_like.csv`, and `config.json` under the chosen output directory.

5.6 Synthetic experiments and validation

We run synthetic experiments to (i) validate masking and permutation invariance/equivariance properties, (ii) reproduce Zhang (2025)'s Table 1-style synthetic example, and (iii) reproduce canonical context effects (decoy, attraction, compromise). Where applicable, we compare TensorFlow outputs to the authors' reference behavior by running the authors-mode path.

Table 1: Four-item synthetic example (analogous to Zhang (2025) Table 1). We report target probabilities and DeepHalo-predicted probabilities for each item.

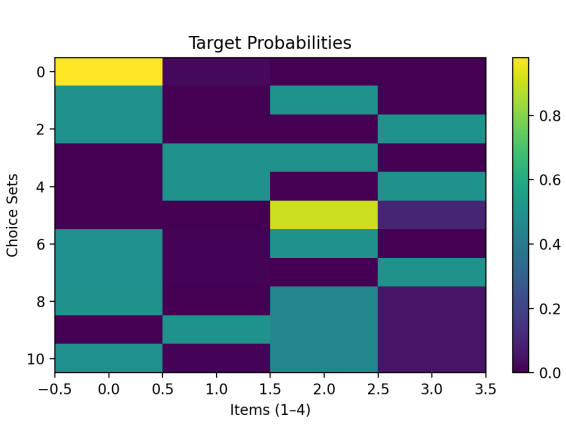
Choice set $\mathcal{C}$	p_1	p_2	p_3	p_4	$\hat{p}_1$	$\hat{p}_2$	$\hat{p}_3$	$\hat{p}_4$
(1, 2)	0.98	0.02	0.00	0.00	0.98	0.02	0.00	0.00
(1, 3)	0.50	0.00	0.50	0.00	0.54	0.00	0.46	0.00
(1, 4)	0.50	0.00	0.00	0.50	0.53	0.00	0.00	0.47
(2, 3)	0.00	0.50	0.50	0.00	0.00	0.52	0.48	0.00
(2, 4)	0.00	0.50	0.00	0.50	0.00	0.44	0.00	0.56
(3, 4)	0.00	0.00	0.90	0.10	0.00	0.00	0.92	0.08
(1, 2, 3)	0.49	0.01	0.50	0.00	0.52	0.01	0.48	0.00
(1, 2, 4)	0.49	0.01	0.00	0.50	0.48	0.01	0.00	0.51
(1, 3, 4)	0.50	0.00	0.45	0.05	0.50	0.00	0.45	0.05
(2, 3, 4)	0.00	0.50	0.45	0.05	0.00	0.49	0.46	0.05
(1, 2, 3, 4)	0.49	0.01	0.45	0.05	0.51	0.01	0.42	0.06

5.6.1 Sanity Checks: Permutation and Masking

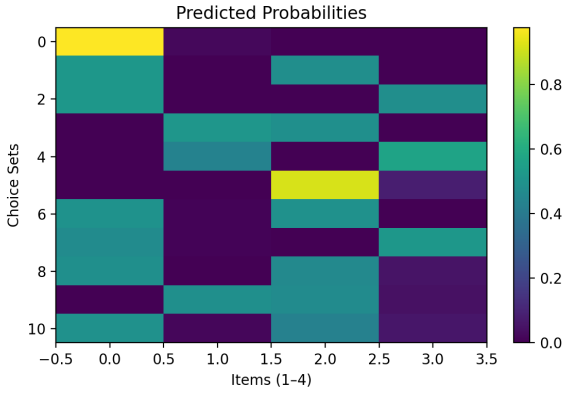
We validate the implementation in three ways. First, we verify permutation invariance by permuting the order of items within each set and confirming that the induced probability vector is unchanged up to the corresponding permutation. Second, we verify masking correctness by checking that unavailable items receive probability zero under the masked softmax. Third, we confirm optimization sanity by overfitting tiny synthetic datasets, driving the negative log-likelihood close to zero, which validates gradient flow and the loss implementation.

5.6.2 Four-Item Synthetic Example (Table 1 Reproduction)

We reconstruct the four-item synthetic example analogous to Table 1 in Zhang (2025)[1]. For each choice set $\mathcal{C}$ with target probability vector $p(S)$, we zero out probabilities for absent items, renormalize over $\mathcal{C}$, and draw 1,000 choices per set, yielding 11,000 observations across 11 distinct choice sets. We train a two-block DeepHalo model ($J = 4$) for 100 epochs with Adam (learning rate 2×10^{-3}). Training reduces NLL from approximately 0.85 to 0.66. Table 1 compares target and predicted probabilities; the mean absolute error is MAE ≈ 0.010 with maximum absolute deviation 0.049, and item rankings are preserved within each set.



(a) Target probabilities.



(b) Predicted probabilities (TensorFlow, simplified path, `authors_mode=False`).

Figure 1: Table 1 reproduction: target vs. predicted choice probabilities across 11 sets.

5.6.3 Decoy Effect

We consider three items A (target), B (competitor), and C (decoy dominated by A), with sets $S_1 = \{A, B\}$ and $S_2 = \{A, B, C\}$. Ground-truth probabilities are specified as:

$$P(A \mid S_1) = 0.45,\ P(B \mid S_1) = 0.55, \qquad P(A \mid S_2) = 0.60,\ P(B \mid S_2) = 0.40,\ P(C \mid S_2) = 0.$$

We generate 5,000 samples per set and train for 80 epochs. The fitted probabilities are:

$$P(A \mid S_1) \approx 0.459, \qquad P(A \mid S_2) \approx 0.599,$$

so the decoy-induced shift is $\Delta_{\text{decoy}} \approx 0.14 > 0$, reproducing asymmetric dominance.

5.6.4 Attraction Effect: TensorFlow vs. Authors' PyTorch

We construct an attraction scenario with A (target), B (dominating competitor), and C (decoy dominated by B), with $S_1 = \{A, B\}$ and $S_2 = \{A, B, C\}$. We specify:

$$P(A \mid S_1) = 0.5,\ P(B \mid S_1) = 0.5, \qquad P(A \mid S_2) = 0.3,\ P(B \mid S_2) = 0.7,\ P(C \mid S_2) = 0.$$

Training both implementations on the same synthetic data yields a nearly identical attraction shift for B:

$$\Delta_B^{\text{TF}} \approx 0.205, \qquad \Delta_B^{\text{Torch}} \approx 0.200,$$

providing cross-framework validation that our TensorFlow reimplementation matches the authors' behavior.

5.6.5 Compromise Effect

We construct a compromise scenario with A (low), B (middle), C (high), with sets $\{A, B\}$, $\{B, C\}$, and $\{A, B, C\}$. The fitted model yields:

$$P(B \mid \{A, B\}) \approx 0.698, \quad P(B \mid \{B, C\}) \approx 0.304, \quad P(B \mid \{A, B, C\}) \approx 0.795.$$

Define the compromise index:

$$\Delta_{\text{comp}} = P(B \mid \{A, B, C\}) - \max\big(P(B \mid \{A, B\}), P(B \mid \{B, C\})\big) \approx 0.097 > 0,$$

confirming a clear compromise effect.

5.6.6 Discussion

Across synthetic experiments, DeepHalo reproduces canonical violations of IIA (decoy, attraction, compromise) and closely matches the authors' PyTorch behavior on attraction. Together with permutation and masking sanity checks, these results provide evidence that our TensorFlow implementation is faithful to the model described in Zhang et al. (2025)[1].

5.6.7 Decoy, Attraction, and Compromise Effects

Figures 2–3 visualize probability shifts under canonical context effects. We include a qualitative influence map (Figure 4) as an interpretability artifact summarizing learned pairwise context effects.

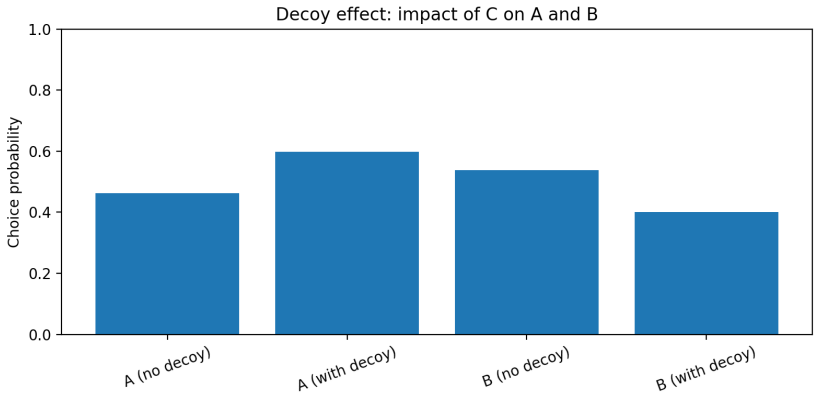
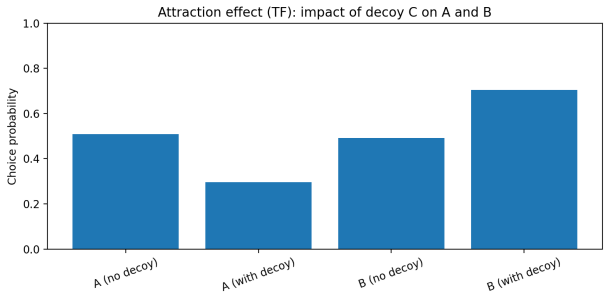
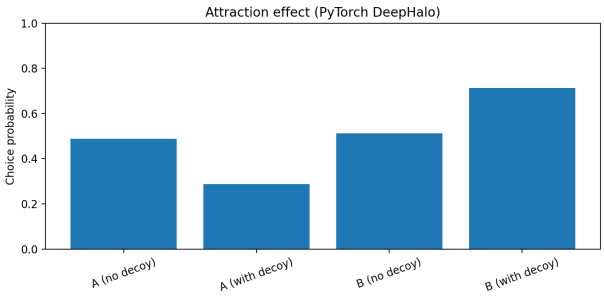


Figure 2: Decoy effect experiment: adding a dominated option shifts probability mass toward the target.



(a) TensorFlow implementation.



(b) Authors' PyTorch reference.

Figure 3: Attraction effect experiment: cross-framework comparison of the probability shift toward the dominating option.

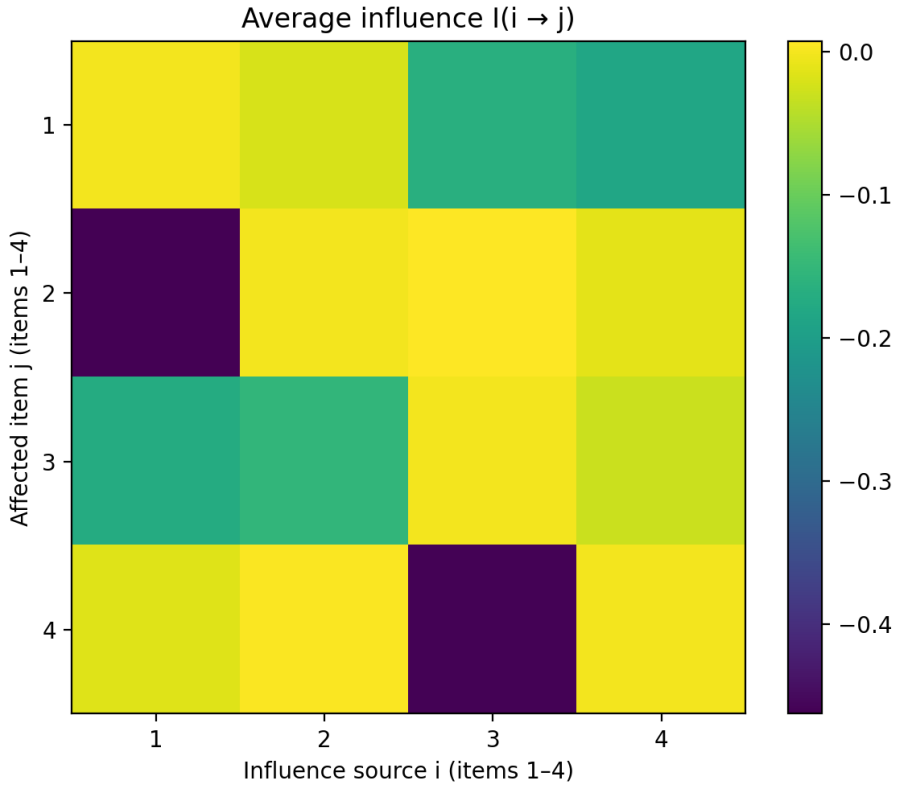


Figure 4: Influence matrix visualization: learned pairwise context effects in the featureless setting.

5.7 Summary of implementation claims for Part 1

The key implementation claim for Part 1 is that we provide an authors-mode TensorFlow port used for replication oriented experiments and a separate simplified permutation-equivariant halo stack used as an ablation. Both paths enforce availability masking via a masked log-softmax. All reported context-effect validations (decoy/attraction/compromise) are evaluated at the probability level, which is the identified object under the model.

6 Verification Tests and Sanity Checks

This section summarizes the validation work we performed to gain confidence that the DeepHalo implementation is correctly wired, respects choice-set structure, and behaves sensibly under masking and menu perturbations. The goal of these checks is not to "prove" correctness formally, but to rule out the most common failure modes in set-based neural choice models: indexing errors, incorrect masking/renormalization, and silent training pathologies.

6.1 Checks executed

Masking and availability. All forward passes use a masked log-softmax so that unavailable items receive effectively $-\infty$ utility and therefore probability exactly zero after normalization. We verified that (i) unavailable items always receive zero probability, and (ii) probabilities renormalize over the offered set. We also verified basic edge cases in the data pipeline, including menus where only the outside option and one inside option are available, and menus where nearly all items are available, to ensure the mask is applied consistently and does not produce numerical instability.

Permutation and indexing sanity checks. DeepHalo is intended to be permutation-equivariant with respect to the ordering of items within a menu, in the sense that permuting the item order (while permuting the corresponding mask and the chosen index consistently) should permute the output probabilities in the same way. We implemented checks that explicitly permute the columns of the menu representation and confirm that predicted probability vectors match up to the same permutation. This test is designed to catch subtle indexing bugs where, for example, halo aggregation uses the wrong axis or masking is applied to a mismatched ordering.

Training and optimization sanity checks. We verified that the model trains without producing NaNs/Infs in logits, log-probabilities, or the negative log-likelihood. On small synthetic datasets, the negative log-likelihood decreases under reasonable hyperparameters, which provides evidence that gradients flow through the full computation graph and that the likelihood is implemented consistently with the masked softmax. This check is especially important for set-based models because a single masking or normalization bug can silently prevent learning while still producing numerically finite outputs.

Consistency across the two implementation paths. Our codebase contains two execution paths controlled by `authors_mode`. When `authors_mode=True`, the top-level `DeepHalo` routes computation through the TensorFlow port of the authors' released architecture (featureless or feature-based). When `authors_mode=False`, it uses a simplified permutation-equivariant halo stack (`BaseEncoder` + `HaloBlock`). We used these two paths as a practical diagnostic: both should respect masking and permutation structure, and both should be able to fit simple synthetic choice patterns. Agreement in qualitative behavior across these paths is not a guarantee of correctness, but it is a useful guardrail against implementation errors that would cause one path to behave inconsistently.

In-sample evaluation scope. All synthetic experiments in Part 1, including the Table 1 reproduction, train on data generated from known target probabilities and evaluate on the same choice sets used for training. There is no held-out test set. This follows the paper's own experimental design — the goal is to verify that the model can *fit* the prescribed menu-dependent probabilities, not to test generalisation to unseen menus. These experiments are fit demonstrations; out-of-sample generalisation in richer settings is discussed in Section 20 and in the alternative model discussion of Section 16.

Cross-framework comparison (when feasible). In settings where we were able to run the authors' reference PyTorch implementation on the same synthetic setup, we compared probability shifts under menu perturbations (e.g., adding a decoy option) and verified that the qualitative trends match. We emphasize that this comparison is qualitative: numerical equality is not expected across frameworks due to different initialization, optimizer implementations, and nondeterminism in low-level kernels. The purpose is to ensure that the direction and qualitative magnitude of canonical context effects are consistent with the reference behavior.

6.2 Additional recommended tests

Several additional tests would strengthen coverage if this were a production system or a longer replication effort. First, identifiability-aware parameter recovery tests should evaluate recovery at the level of probabilities (or probability ratios) rather than raw utilities, because softmax utilities are only identified up to set-specific additive constants.

Second, counterfactual stability tests should systematically perturb menus (add/remove dominated options, swap availability patterns, vary menu size) and verify that predicted probability shifts are stable across random seeds. Third, sensitivity tests should quantify how results vary with initialization, depth/width, and dropout to diagnose overfitting risk on sparse menu data. Finally, calibration and robustness checks (for example, reliability plots on held-out menus, or stress tests under distribution shifts in menu composition) would be valuable in a real credit-card offer setting where menus and targeting rules evolve over time.

7 Comparison with Yang (2025) RKHS Choice Model

Zhang (2025) and Yang (2025) share the same high-level motivation: the standard multinomial logit imposes Independence of Irrelevant Alternatives (IIA), which fails in the presence of context effects such as attraction, decoy, compromise, and more general menu-dependent substitution patterns. Both papers therefore allow utilities (or utility *components*) to depend on the full offered set S rather than only on item attributes. The approaches diverge sharply in (i) the function class used to represent menu-dependence, (ii) how regularization is imposed, (iii) optimization and statistical guarantees, and (iv) practical scalability for large choice universes such as credit-card offers.

7.1 Problem setup and what it means to "relax IIA"

In both frameworks, the data consist of pairs (S_i, y_i) where $S_i \subseteq \{1, \dots, J\}$ is the offered menu and $y_i \in S_i$ is the chosen item (with an optional outside option). A generic menu-dependent logit writes

$$P(y = j \mid S) \;=\; \frac{\exp(u_j(S))}{\sum_{k \in S} \exp(u_k(S))}, \tag{2}$$

so relaxing IIA is equivalent to allowing the systematic utilities $u_j(S)$ to change when S changes, even for a fixed focal item j.

A useful way to interpret "context effects" is through pairwise and higher-order differences. For example, the change in the log-odds for j versus k under a menu perturbation $S \mapsto S \cup \{\ell\}$ is

$$\Delta_{jk}(\ell; S) \;\triangleq\; \log \frac{P(j \mid S \cup \{\ell\})}{P(k \mid S \cup \{\ell\})} \;-\; \log \frac{P(j \mid S)}{P(k \mid S)} \;=\; \big(u_j(S \cup \{\ell\}) - u_k(S \cup \{\ell\})\big) - \big(u_j(S) - u_k(S)\big). \tag{3}$$

Under IIA (standard MNL), $u_j(S)$ does not depend on S, so $\Delta_{jk}(\ell; S) = 0$ for all j, k, ℓ, S. Both Zhang and Yang permit $\Delta_{jk}(\ell; S) \neq 0$, but they impose different structural restrictions on *how* S enters.

7.2 Modeling philosophy: learned architecture versus chosen kernel

DeepHalo (Zhang, 2025): neural set-to-utilities mapping. DeepHalo represents menu dependence by learning a permutation-equivariant transformation of the set (or of item embeddings conditioned on the set). In the featureless setting, the learned object can be viewed abstractly as a family of functions

$$u_j^{\mathrm{DH}}(S) \;=\; f_\theta(j, S), \qquad \theta \in \Theta, \tag{4}$$

implemented by repeated "halo" blocks that update an item representation using a symmetric aggregate of the menu. In our implementation, this appears in two concrete forms:

1. In `authors_mode=True`, we route computation through `AuthorsFeaturelessNetTF` / `AuthorsFeatureBasedNetTF`, which are a TensorFlow port of the authors' released PyTorch design. This path encodes menu dependence via head-wise transforms and multiplicative interactions between an item-specific transformation and an aggregated context signal, followed by a masked log-softmax.
2. In `authors_mode=False`, we use a simplified but still permutation-equivariant halo stack built from `BaseEncoder` and repeated `HaloBlocks`. Each block computes a masked mean context vector and applies head-wise MLP updates to $[z_{bj} \,\|\, c_b]$ with a residual update and layer normalization.

In both cases, the inductive bias is that menu effects are captured through repeated, shared blocks that respect permutation equivariance and availability masking.

RKHS choice model (Yang, 2025): function-space regularization. Yang's approach begins by selecting a (matrix-valued) positive definite kernel on sets, and then searching for utilities in the corresponding vector-valued RKHS. Abstractly, one can write

$$u^{\mathrm{RKHS}}(\cdot) \in \mathcal{H}, \qquad u^{\mathrm{RKHS}}(S) \in \mathbb{R}^J \tag{5}$$

where $\mathcal{H}$ is a vector-valued Hilbert space determined entirely by the chosen kernel $K(\cdot, \cdot)$. A convenient representation is

$$u^{\mathrm{RKHS}}(S) \;=\; K(\cdot, S)^\top \alpha, \tag{6}$$

where the Representer Theorem implies the learned function can be expressed in terms of kernel evaluations at observed menus. The key philosophical difference is that DeepHalo learns a representation end-to-end with relatively weak explicit prior structure beyond architecture, while RKHS methods encode structure *explicitly* through the kernel choice and regularization.

7.3 Capacity control and what "complexity" means in each model

DeepHalo: implicit capacity control via architecture and training. For DeepHalo, complexity is controlled primarily by design choices such as embedding dimension d, number of blocks L, number of heads H, and width of internal MLPs/residual layers. Regularization typically enters through early stopping, weight decay, dropout, or other training heuristics. Even when explicit regularizers are used, the effective complexity is often governed by the interaction of stochastic optimization and overparameterization, i.e., implicit bias.

A practical implication is that DeepHalo can represent highly nonlinear menu effects, but understanding "how complex" the fitted function is requires diagnostic work (sensitivity to depth/width, seed stability, and generalization evaluation).

RKHS: explicit norm-based capacity control. In the RKHS formulation, capacity control is explicit and interpretable. The estimator solves a regularized empirical risk minimization problem

$$\min_{u \in \mathcal{H}} \frac{1}{N} \sum_{i=1}^{N} \ell\big(y_i, u(S_i)\big) + \lambda \|u\|_{\mathcal{H}}^2, \tag{7}$$

where ℓ is the negative log-likelihood induced by (2). The penalty $\|u\|_{\mathcal{H}}^2$ provides a direct, theoretically grounded notion of smoothness/complexity relative to the kernel. The parameter λ is an explicit knob controlling under/overfitting in a way that is often easier to interpret than "increase depth" or "increase width."

7.4 Optimization landscape and statistical guarantees

DeepHalo: nonconvex likelihood. DeepHalo is trained by minimizing the negative log-likelihood

$$\min_{\theta \in \Theta} \; -\sum_{i=1}^{N} \log P_\theta(y_i \mid S_i), \tag{8}$$

which is nonconvex in θ. As a result, optimization guarantees are local, and training outcomes can vary with initialization, optimizer settings, and stochasticity. In practice, DeepHalo can still be very effective, but the burden shifts to good engineering: reproducibility controls, ablations, and evaluation under held-out menu perturbations.

RKHS: convexity (for fixed kernel) and representer structure. For a fixed kernel and a convex surrogate (the logit negative log-likelihood is convex in the function values but the full functional problem is handled through the RKHS machinery), the RKHS approach yields a well-posed regularized problem. The representer theorem reduces the infinite-dimensional optimization to a finite-dimensional one in the coefficients of the kernel expansion. This delivers global optimality for the reduced problem and avoids local minima issues typical in deep networks. The tradeoff is that the kernel must be chosen a priori, and the computational cost can become significant without approximation.

7.5 Expressiveness: learning interactions versus encoding interactions

DeepHalo: adaptive interaction learning. DeepHalo learns interaction structure from data. Depth and repeated halo updates allow the model to build higher-order dependencies: a new option can change the representation of every other option through shared context aggregation and nonlinear transforms. In the featureless case with finite J, increasing depth/width increases the set-function capacity substantially, and the model can approximate a wide range of menu-dependent choice rules.

RKHS: interaction structure is determined by the kernel. In the RKHS approach, the kernel largely determines what kinds of menu perturbations are "easy" or "hard" for the model. For example, a kernel that depends only on first-order summaries of S will encode predominantly low-order context effects, whereas a richer kernel (e.g., capturing pairwise co-occurrence or higher-order set interactions) can represent more complex substitution patterns. This means RKHS modeling shifts the modeling burden from network design/training to kernel design: one must decide which invariances and interaction orders are appropriate, and then trust that the regularized fit finds the best function within that chosen space.

7.6 Scalability and computational tradeoffs

DeepHalo: minibatch-friendly and naturally large-scale. DeepHalo training scales roughly with the parameter count and the per-batch cost of halo updates. Because it is trained with stochastic optimization, memory usage can be approximately linear in batch size rather than in the dataset size N. This is attractive for settings like credit-card offers where (i) N can be large, (ii) menus evolve frequently, and (iii) new items/offers appear over time.

RKHS: kernel methods can be quadratic without approximations. A direct RKHS implementation requires evaluating and storing kernel matrices over observed menus, which is typically $O(N^2)$ in memory and time. Yang's approach motivates approximation techniques (random features, Nyström methods, or neural tangent kernel approximations) to reduce this burden. These approximations can make RKHS models viable at larger scales, but introduce approximation error and additional hyperparameters (number of features, approximation rank), and often require careful tuning to remain stable.

7.7 Interpretability and diagnostics

DeepHalo: flexible but less transparent. DeepHalo can produce interpretable *behavioral* artifacts (probability shifts under specific menu perturbations), but attributing those shifts to a small set of parameters is difficult because effects are distributed across embeddings and multiple nonlinear layers. In practice, interpretability often relies on post-hoc analysis: influence matrices, gradient-based sensitivity, or controlled counterfactual experiments.

RKHS: clearer separation between assumptions and fit. The RKHS framework separates (i) the modeling assumption (the kernel K), (ii) the capacity control (the RKHS norm penalty $\lambda\|u\|_{\mathcal{H}}^2$), and (iii) the fitted coefficients. This separation can make it easier to articulate what the model can and cannot represent and to diagnose overfitting through λ and kernel complexity. At the same time, interpretability is only as good as the kernel design: if the kernel embeds complex co-occurrence structure, the fitted function can still be hard to explain, but the source of complexity is explicit rather than implicit.

7.8 Implications for credit-card offer demand estimation

For credit-card merchant offers, both approaches have plausible roles, but their strengths are different.

DeepHalo is attractive when menus are large and heterogeneous, when context effects plausibly arise from recommendation and ranking systems (which can create complicated substitution patterns), and when the primary goal is accurate prediction under menu changes. It is also operationally convenient because it trains with minibatches and can be updated frequently as new offers enter the system.

The RKHS approach is attractive when one wants stronger optimization guarantees and explicit control of complexity, particularly if the analyst can propose a kernel that encodes domain structure (for example, kernels that reflect merchant-category similarity, offer-term similarity, or co-exposure patterns). In contexts where interpretability and stable estimation are paramount, a well-chosen kernel with norm regularization can be easier to defend than a deep architecture trained nonconvexly.

In summary, Zhang (2025) provides an adaptive, data-driven mechanism for learning menu dependence with strong scalability properties, while Yang (2025) provides a principled function-space approach whose guarantees and interpretability hinge on kernel choice and whose scalability hinges on approximation methods. In a realistic credit-card offer setting, the practical decision is likely driven by data size and operational constraints (favoring DeepHalo) versus the availability of credible kernel structure and the need for convex optimization and explicit regularization (favoring RKHS).

8 Suitability for Credit-Card Offer Demand Estimation

In the credit-card merchant-offer setting, each alternative is an offer with observable terms such as the redemption amount (or implied value), required spend threshold, duration, merchant category, eligibility constraints, and (for credit products) APR/fees. Consumers typically see a subset of offers at a given time, and the bank's operational goal is to predict take-up and perform counterfactuals, such as how redemption value or eligibility rules affect demand, or how adding/removing an offer changes substitution patterns. DeepHalo (Zhang, 2025) and the RKHS choice model (Yang, 2025) are both capable of capturing menu dependence, so they can be useful for describing and predicting context effects. The key question is whether they can be used as *structural* demand estimators for policy and design decisions in a setting where offer assignment and offer terms are not randomly generated.

A central obstacle is that both the menu S and the terms inside the menu are typically chosen endogenously. In practice, banks determine what offers appear to a user using targeting rules based on credit score, spend history, risk segmentation, merchant partnerships, and predicted uplift. Vendors also co-fund campaigns, which can shift both the attractiveness of an offer and the bank's incentives to display it. This means that the probability model being learned is generally of the form

$$P(y = j \mid S, X),$$

where X denotes observed features, but S and important elements of X can be correlated with unobserved determinants of utility. Without additional identifying assumptions (valid instruments, a control-function structure, experimental variation, or structural restrictions on unobservables), DeepHalo and RKHS methods learn *associations* that mix preference and policy. In that case, estimated substitution patterns can reflect the targeting and ranking system rather than stable underlying consumer preferences, and counterfactuals that change the policy (what is shown, to whom, and when) can be biased.

Counterfactual stability is the second major concern. Demand estimation for offer design requires predictions under menu and attribute configurations that may not have been observed historically, including new offers, new merchants, new eligibility regimes, and changes in redemption generosity. DeepHalo is highly expressive and can interpolate well when the counterfactual stays close to the training distribution, but neural set models can behave unpredictably outside that support. The RKHS approach imposes smoothness and explicit regularization through the RKHS norm, which can improve stability in some settings, but it still does not guarantee valid extrapolation when the policy shift is large, because the kernel is a modeling choice and the data may not identify utility responses in unobserved regions of the menu space. In both cases, the safest interpretation is predictive: these models can be strong forecasters for *similar* menus to those observed, but their structural validity under large interventions must be defended with explicit assumptions or external variation.

A third issue is interpretability and the ability to produce decision-relevant elasticities. Classical structural models (e.g., logit and random-coefficients logit) parameterize the effect of offer terms directly and can deliver interpretable marginal utilities and price-like elasticities. DeepHalo distributes menu interactions across embeddings and nonlinear blocks, so interpreting a single parameter as a willingness-to-pay is not generally possible without additional structure or post-processing (for example, computing local derivatives of predicted choice probabilities with respect to a numeric attribute). RKHS models provide a clearer separation between assumptions (the kernel) and regularization (the RKHS norm), but the learned coefficients still represent a function expansion rather than a small set of structural parameters. In regulated financial environments, where explanations and governance often matter, this reduces the attractiveness of purely predictive menu-dependent models as standalone tools for pricing or major policy design.

Data requirements also matter. DeepHalo typically benefits from large and diverse datasets to learn higher-order interactions robustly. In credit-card offers, take-up events can be sparse, menus can change quickly, and certain menu configurations may be rare because targeting policies constrain what is shown. This creates a risk that a high-capacity model overfits historical targeting patterns unless the analyst uses careful regularization, strong validation under menu perturbations, and robustness checks across time windows and random seeds. RKHS methods can be statistically efficient in smaller samples when the kernel is well-specified, but exact kernel methods scale poorly with the number of observations unless approximations (random features, Nyström) are used, and approximation choices introduce their own tuning and failure modes.

Overall, DeepHalo and RKHS choice models are well-suited as tools for *measuring and predicting* context dependence in observed offer menus, and they can be valuable for operational forecasting and for diagnosing menu effects (e.g., attraction/decoy-like behavior in the platform). For *structural* demand estimation in credit-card offers, where endogeneity, interpretability, and policy-counterfactual validity are central, these models typically need to be paired with additional identification strategies. In particular, they become more defensible when combined with (i) randomized experiments or quasi-experimental variation in offer display, (ii) explicit modeling of unobserved shocks and endogeneity corrections as in Lu & Shimizu (2025), or (iii) structural restrictions that separate stable preferences from the bank's targeting policy. Under those additions, context-dependent models can play a

complementary role: they can capture realistic substitution patterns once identification of price-like effects and unobservables is addressed by the broader econometric design.

9 Alternative Models Worth Considering

The discussion above highlights three recurring requirements for credit-card offer demand estimation: endogeneity must be handled explicitly, counterfactuals must be stable under policy changes, and model outputs must often be interpretable enough to support governance and decision-making. These constraints do not rule out DeepHalo or other menu-dependent models, but they do suggest that a practical modeling workflow should include additional baselines that either (i) impose stronger economic structure or (ii) deliver reliable predictive performance on tabular data with clearer diagnostics. This section summarizes several model families that are particularly relevant in a credit-card offer setting and clarifies what each approach buys, what assumptions it needs, and what it cannot do on its own.

9.1 Structural econometric models

Random-coefficients (mixed) logit. A widely used structural benchmark is the random-coefficients logit model, which relaxes IIA through taste heterogeneity rather than through direct menu dependence. Utility for consumer i choosing offer j in market t is written as

$$u_{ijt} \;=\; x_{jt}^{\top}\beta_i \;-\; \alpha_i p_{jt} \;+\; \xi_{jt} \;+\; \varepsilon_{ijt}, \qquad (\beta_i, \alpha_i) \sim F(\theta), \tag{9}$$

with ε_{ijt} typically assumed i.i.d. Type-I extreme value. Relative to a simple logit, the mixing distribution $F(\theta)$ generates flexible substitution patterns because different consumers respond differently to attributes and price-like terms. The key advantage of this approach is interpretability: α_i and β_i correspond directly to marginal utilities, and the model naturally delivers elasticities and welfare measures. The main difficulty in a credit-card offer setting is the same as in BLP: if p_{jt} is endogenous because banks and vendors set offer terms in response to unobserved demand drivers, then identification requires instruments, a control-function correction, experimental variation, or structural assumptions on ξ_{jt} (such as the sparsity restriction studied in Part 2). When those identification conditions are credible, mixed logit is often the cleanest way to connect estimated demand to counterfactual design decisions.

Nested and generalized extreme value (GEV) models. Nested logit and related GEV models impose structured correlation across alternatives by grouping similar offers into nests (e.g., "premium travel cards," "cash-back cards," "merchant-category offers"). A canonical nested logit writes utility as

$$u_{ijt} \;=\; x_{jt}^{\top}\beta \;-\; \alpha p_{jt} \;+\; \xi_{jt} \;+\; \eta_{ig(j)} \;+\; \varepsilon_{ijt}, \tag{10}$$

where $g(j)$ indexes the nest and η_{ig} induces correlation across offers in the same nest. These models preserve closed-form choice probabilities and are computationally lighter than random-coefficients logit, while still allowing richer substitution than IIA. Their limitation is that the substitution structure is restricted by the nesting design, so they are best used when domain knowledge supports a meaningful hierarchy (for example, category-level similarities in merchant offers). As with mixed logit, endogeneity in p_{jt} requires instruments or other identification strategies if the goal is structural inference rather than prediction.

Latent-class (finite-mixture) choice models. Latent-class models treat the population as a discrete mixture of preference types:

$$P(y = j \mid S) \;=\; \sum_{c=1}^{C} \pi_c \cdot \frac{\exp\!\left(x_j^{\top}\beta_c - \alpha_c p_j\right)}{\sum_{k \in S} \exp\!\left(x_k^{\top}\beta_c - \alpha_c p_k\right)}, \qquad \sum_{c=1}^{C} \pi_c = 1. \tag{11}$$

This approach captures segmentation in a way that is often operationally meaningful in finance (e.g., "revolvers" versus "transactors," or customers who prioritize travel rewards versus cash-back). It is typically easier to fit and explain than a continuous random-coefficients model, and it can be a strong baseline when the dataset is not large enough to support high-capacity neural menu models. The tradeoff is that heterogeneity is discretized and the number of classes C becomes a substantive modeling choice that affects both fit and interpretation.

9.2 Semi-parametric and hybrid approaches

Partially linear or hybrid utility models. A common compromise between interpretability and flexibility is to retain an explicit parametric component for key economic variables (especially price-like terms) while allowing other components to be flexible:

$$u_{ijt} \;=\; g_{\theta}(x_{jt}) \;-\; \alpha p_{jt} \;+\; \varepsilon_{ijt}, \tag{12}$$

or, more generally,

$$u_{ijt} \;=\; g_\theta(x_{jt}) \;+\; z_{jt}^{\top}\alpha \;+\; \varepsilon_{ijt}, \tag{13}$$

where $g_\theta(\cdot)$ may be a neural network or tree-based function and z_{jt} contains variables for which one wants stable and interpretable marginal effects. This formulation is attractive in credit-card offers because it allows the model to learn complex interactions among offer attributes while maintaining a clear interpretation for selected policy levers (APR, fees, reward rate, required spend). The main caution is that endogeneity is not automatically solved by flexibility: if the policy levers are correlated with unobserved shocks, then α can still be biased unless one introduces instruments, a control-function correction, experimental variation, or explicit structure on the unobserved component.

Tree-based predictive baselines (XGBoost/LightGBM). For operational prediction tasks (e.g., ranking offers for a user or forecasting take-up under the current targeting policy), gradient-boosted decision trees are often strong baselines on tabular data. A typical setup treats each offer impression as an observation and predicts take-up probability using engineered features and interactions. While these models do not produce a structural utility function, they provide practical advantages: they handle nonlinearities and high-order interactions without heavy architecture design, they train quickly, and they can be diagnosed with standard tools such as feature importance or SHAP. In a credit-card offer setting, they can be useful as a "best-in-class" predictive comparator to evaluate whether the complexity of a menu-dependent neural choice model is justified for the forecasting objective. However, their counterfactual validity is limited when the counterfactual changes the policy that generated the training data, and they are not structural without additional causal design.

9.3 Models explicitly addressing endogeneity

Because offer terms and offer display are typically endogenous, methods that explicitly address endogeneity are often necessary when the objective is causal demand estimation rather than prediction under the status quo. One option is a control-function approach, where the analyst specifies a first-stage model for the endogenous term and then introduces the first-stage residual as an additional regressor in utility to correct correlation with the unobserved shock. Another option is double/debiased machine learning, which uses flexible nuisance models but targets a lower-dimensional causal parameter under orthogonality conditions. A third option, directly relevant to Part 2, is to impose structural restrictions on the unobserved component (e.g., Lu & Shimizu's sparsity assumption) and estimate it jointly with preferences. These approaches differ in the assumptions they require, but they share a key property: they explicitly separate "preference" from "policy" or "pricing" channels, which is precisely what purely predictive menu-dependent models do not guarantee.

9.4 Overall assessment

DeepHalo and RKHS models are valuable for capturing menu dependence and can be strong predictive tools when the primary goal is forecasting under observed menu policies. For structural demand estimation of credit-card offers, traditional econometric models remain important because they provide interpretable parameters and well-understood counterfactual machinery, and they can be paired with explicit endogeneity corrections. In practice, a robust workflow is to use a structured model (mixed logit, nested logit, or latent-class) as the interpretability and policy baseline, use flexible predictive models (boosted trees, neural nets) as forecasting comparators, and incorporate endogeneity corrections (IV/control-function or sparse-shock structure) whenever the goal is a causal statement about how changing offer terms will change demand.

Part II
Market–Product Demand Shocks (Lu & Shimizu 2025)

10 Overview and Problem Formulation

10.1 Motivation: Endogeneity and Unobserved Market–Product Characteristics

In the credit card merchant-offer setting, consumers face menus of offers whose terms have explicit dollar value (e.g., "spend x then get y back"). A discrete choice model maps offer attributes and consumer/context features into choice probabilities, enabling counterfactual forecasting for campaign design. A key empirical challenge is that offer terms (including price-like variables such as APR/fees, required spend thresholds, or effective cost of redemption) are often *endogenously* chosen by the bank and co-funding vendor. In particular, offer generosity and targeting intensity are influenced by information that may not be observed to the analyst (e.g., internal uplift models, underwriting constraints, vendor subsidy schedules, channel intensity). This leads to correlation between observed offer attributes and *unobserved* demand components, biasing naive estimators. Part 1 focused on *context effects* (menu dependence). Part 2 focuses on a different but complementary misspecification: *market–product demand shocks*, also called unobserved market–product characteristics.

10.2 Random Utility Model with Market–Product Shocks

Let markets be indexed by $t \in \{1, \dots, T\}$ (e.g., time × segment × channel), and products/offers in market t by $j \in \{1, \dots, J_t\}$, with outside option $j = 0$. Let $x_{jt} \in \mathbb{R}^K$ denote observed offer covariates and let p_{jt} denote a price-like variable. We observe either (i) individual choices $y_{it} \in \{0, 1, \dots, J_t\}$ for consumers $i = 1, \dots, N_t$, or (ii) aggregated counts/shares.

We write utility as

$$u_{ijt} \;=\; \delta_{jt} + \mu_{ijt} + \varepsilon_{ijt}, \qquad \varepsilon_{ijt} \overset{i.i.d.}{\sim} \text{Type-I EV}, \tag{14}$$

where μ_{ijt} captures observed heterogeneity (random coefficients) and δ_{jt} is the mean utility:

$$\delta_{jt} \;=\; x_{jt}'\beta \;-\; \alpha p_{jt} \;+\; \xi_{jt}. \tag{15}$$

The term ξ_{jt} is an *unobserved market–product demand shock* (unobserved product-market characteristic). It captures unrecorded promotional exposure, targeting intensity, vendor co-funding intensity, or latent product appeal in market t.

10.3 Endogeneity and the IV Benchmark

If the bank and vendor choose p_{jt} (or offer generosity) using information correlated with ξ_{jt}, then

$$\mathbb{E}[\xi_{jt} \mid p_{jt}, x_{jt}] \neq 0, \tag{16}$$

and regressing δ_{jt} on (x_{jt}, p_{jt}) yields biased (β, α). The classical benchmark approach (BLP + IV) introduces instruments z_{jt} satisfying

$$\textbf{Relevance:} \quad \mathrm{Cov}(z_{jt}, p_{jt}) \neq 0, \tag{17}$$

$$\textbf{Exclusion:} \quad \mathbb{E}[z_{jt}\xi_{jt}] = 0. \tag{18}$$

In many marketing and credit settings, exclusion is difficult to defend because plausible cost shifters can also correlate with unobserved campaign intensity.

10.4 Lu(25) Perspective: Modeling ξ_{jt} via Structural Sparsity

Lu(25) proposes to handle ξ_{jt} by imposing *structure* rather than relying solely on instruments. A key modeling assumption is that market–product shocks are *concentrated*: only a small fraction of product–market pairs experience large shocks (e.g., episodic vendor-funded promotions or targeted exposure).

A common decomposition is

$$\xi_{jt} \;=\; \nu_t \;+\; \gamma_{jt}, \tag{19}$$

where ν_t is a market-level baseline shock and γ_{jt} is an idiosyncratic market–product deviation that is sparse across j (and potentially across t). Estimation proceeds by (i) inverting shares to recover mean utilities and (ii) estimating (β, α) jointly with a regularized/shrinkage model for γ_{jt}.

10.5 Implementation Note (TF/TFP) and Roadmap

In our implementation, we use TensorFlow for numerically stable computation and TensorFlow Probability (`tfp.mcmc`) to construct an MCMC procedure for the shrinkage model. Because TFP does not provide a built-in Gibbs sampler for discrete inclusion variables, our implementation uses an MCMC strategy compatible with `tfp.mcmc`. The specific kernel choices and any deviations from Lu & Shimizu (2025) are documented in Part 2(b), alongside replication outputs and diagnostics.

11 Literature Review

11.1 Problem formulation: endogenous offer terms and unobserved market–product shocks

The central econometric problem in Part 2 is identification of demand parameters when observed offer terms are correlated with unobserved market–product characteristics. In a random utility model for market t and product/offer j, mean utility is typically written as

$$\delta_{jt} = x_{jt}'\beta - \alpha p_{jt} + \xi_{jt}, \tag{20}$$

where p_{jt} is a price-like attribute, x_{jt} are observed offer attributes, and ξ_{jt} is an unobserved market–product demand shock. In the credit-card merchant-offer setting, a "price-like" variable can be operationalized as any scalar that encodes how costly or attractive an offer is to a consumer, such as APR/fees, a required spend threshold, or an effective net cost (or generosity index) implied by "spend x then get y back." The key concern is that the bank and vendor choose offer terms using information that is not fully observed to the analyst, such as internal uplift scores, channel exposure decisions, underwriting constraints, or vendor subsidy rules. These latent inputs affect both take-up and offer design, so ξ_{jt} is correlated with p_{jt} even after conditioning on x_{jt}, which implies $\mathbb{E}[\xi_{jt} \mid p_{jt}, x_{jt}] \neq 0$. This endogeneity problem is conceptually distinct from the context-dependence problem studied in Part 1. Context-dependent choice models relax IIA and allow the menu to affect utilities, but they do not automatically restore exogeneity of p_{jt}. A deep model can still fit correlations created by the joint optimization of offer terms and exposure, and those correlations can distort demand responses if they are interpreted causally. Part 2 therefore focuses on methods that address the unobserved component ξ_{jt} (and the resulting bias) in addition to any menu effects.

11.2 BLP and the classical IV benchmark

A standard approach to price endogeneity in differentiated-products demand is the Berry–Levinsohn–Pakes (BLP) framework, which combines a random-coefficient logit demand system with moment conditions using instruments for price. The canonical references are Berry (1994) and Berry, Levinsohn, and Pakes (1995).[1] In BLP-style models, one inverts observed market shares to recover mean utilities δ_{jt} for a given heterogeneity parameterization, and then estimates (β, α) by GMM under moment restrictions of the form $\mathbb{E}[z_{jt}\xi_{jt}] = 0$ for valid instruments z_{jt}. The inversion step (often implemented via a contraction mapping) is a computational identity that recovers δ_{jt} from observed shares, while IV/GMM is the identification device that breaks the correlation between p_{jt} and ξ_{jt}. The identifying content of BLP therefore rests on two conditions. First, instruments must be *relevant* in the sense that they shift p_{jt} conditional on other covariates. Second, instruments must satisfy an *exclusion* restriction in the sense that they are orthogonal to ξ_{jt} in the population. The exclusion requirement is particularly demanding in marketing contexts because plausible cost shifters can also co-move with latent promotional intensity or targeting. For credit-card offers, vendor subsidy schedules, channel budgets, and targeting policies can jointly determine both offer terms and latent appeal. As a result, even when the share inversion is computed accurately, IV-based estimates can be biased if exclusion is only approximately satisfied, and they can be unstable if instruments are weak or highly collinear. From a practical perspective, the BLP benchmark remains attractive because it yields interpretable elasticities and supports counterfactual simulation for campaign design. Its main fragility is that the analyst must argue for instruments that move offer terms but not unobserved demand shocks, which can be difficult when offer terms are optimized using proprietary signals that are correlated with take-up.

11.3 Endogeneity corrections beyond classical IV

When exclusion is difficult to defend, the literature offers alternative strategies that modify either the model for p_{jt} (and use that model to control for endogeneity) or the model for ξ_{jt} (and impose structure on the unobservables). One broad class is control-function methods. These approaches specify a reduced-form or structural equation for p_{jt} and include a control residual (or a flexible function of it) in utility so that conditional on the control, the remaining variation in price is exogenous. The identifying assumption is that the control captures the part of p_{jt} correlated with ξ_{jt}, restoring orthogonality after conditioning. This can be attractive when instruments are available for the price equation but exclusion is more plausible in the price equation than directly in the demand equation. The weakness is that the control-function structure must be correctly specified in the sense relevant for identification, and in offer-design settings the pricing/term-setting process can depend on complex internal

[1] If your bibliography file does not yet include entries for Berry (1994) and BLP (1995), add BibTeX keys (e.g., `berry1994` and `blp1995`) and cite them here.

optimization objectives that are hard to model explicitly.[2] A second class uses proxies for unobserved demand shocks. Proxy-variable approaches assume that an observed proxy is informative about ξ_{jt} and that conditional on the proxy, the residual component is orthogonal to p_{jt}. In marketing settings, impressions, targeting scores, and channel intensity logs can sometimes function as proxies, but credibility depends on whether the proxy is sufficiently rich and whether it is not itself contaminated by the same endogeneity channels that drive p_{jt}. The practical risk is that proxies often partially measure campaign intensity but do not capture the full latent state, leaving residual endogeneity. A third class uses latent-factor or random-effects structure. Instead of assuming that shocks are sparse, one can posit that ξ_{jt} has low effective dimension, such as a factor model $\xi_{jt} \approx a_t' b_j$ for low-dimensional a_t and b_j. This reduces the number of free parameters relative to an unrestricted ξ_{jt} and can stabilize estimation when markets and products are numerous. The tradeoff is that low-rank structure implies broadly shared latent components rather than rare, localized shocks. In offer settings where deviations may be episodic and concentrated (for example, a limited-time vendor-funded subsidy that affects only a subset of merchants), sparsity may be more plausible than low-rank structure. Regularization-based approaches treat components of ξ_{jt} as parameters but restrict them via penalties (e.g., ℓ_1) or hierarchical priors. These methods are conceptually close to Lu & Shimizu (2025) because they replace a hard exclusion restriction with an explicit structural assumption about the unobservables. The main empirical burden shifts from arguing for instrument validity to defending the structural restriction (such as sparsity) and demonstrating robustness to prior or penalty strength.

11.4 Lu & Shimizu (2025): sparse market–product shocks as a structural restriction

Lu & Shimizu (2025) propose to model market–product shocks directly and exploit sparsity in their support rather than relying solely on instruments.[2] Their motivating observation is that in many applications only a minority of market–product pairs experience large idiosyncratic shocks because major deviations are driven by discrete events such as targeted campaigns, vendor-funded discounts, channel outages, or episodic exposure changes. This motivates the decomposition

$$\xi_{jt} = \mu_t + d_{jt}, \qquad d_{jt} \text{ is sparse across } (j,t), \tag{21}$$

where μ_t captures a market-level component and d_{jt} captures deviations that are concentrated on a small subset of market–product pairs. Under this perspective, the analyst does not attempt to remove ξ_{jt} using instruments. Instead, the analyst estimates ξ_{jt} (or its sparse component) jointly with the preference parameters under a sparsity-inducing prior (or penalty), typically using MCMC in spike-and-slab formulations. Relative to IV-based identification, Lu & Shimizu (2025) replace an exclusion restriction with a structural restriction on the unobservables: identification is driven by the assumption that d_{jt} is sparse (or approximately sparse) and that the likelihood contains enough information to separate systematic utility from rare, large shocks. This shift is attractive in settings where exclusion is hard to defend, but it introduces sensitivity to the sparsity prior (or penalty) and to posterior computation. A credible application therefore benefits from diagnostics that show the sparsity mechanism is active, such as posterior inclusion probabilities separating signal and noise, and robustness checks over prior scales and MCMC tuning. The approach also provides an operational diagnostic benefit that IV does not naturally provide. Because d_{jt} is explicitly estimated, one can rank market–product pairs by inferred shock magnitude (or by posterior inclusion probability) and identify specific offers or markets where latent shocks dominate fit. In a merchant-offer context, such diagnostics can flag campaigns whose outcomes are driven by unobserved targeting or exposure shifts, which can motivate improved instrumentation, better logging, or richer feature sets.

11.5 Relationship between Lu(25) shrinkage and BLP benchmarks in simulation studies

Simulation studies, such as Section 4 of Lu & Shimizu (2025), are designed to isolate conditions under which IV-based estimation fails and sparse-shock estimation succeeds.[2] The key comparison is not only predictive fit but also recovery of unobserved shocks and stability of price sensitivity under endogeneity. In IV-based benchmarks, identification requires that instruments move p_{jt} while remaining orthogonal to ξ_{jt}. In sparse-shock estimation, identification instead relies on the structural restriction that d_{jt} is sparse and that the model is not saturated by unrestricted unobservables. These strategies can be viewed as complementary rather than mutually exclusive. In environments where strong cost shifters exist and exclusion is credible, BLP+IV remains a compelling benchmark for interpretability and tradition. In environments where exclusion is fragile or instruments are weak, modeling ξ_{jt} directly under structural restrictions can be preferable. The credit-card offer setting plausibly contains both regimes: some offer terms may be driven by clear cost mechanisms (interchange, vendor subsidy schedules, regulatory constraints), while other terms may be optimized using latent targeting signals that are difficult to observe externally.

[2] If you include control-function methods in the report, it is standard to cite an econometrics reference (e.g., Rivers and Vuong, 1988; Blundell and Powell, 2004) and then explain what "price equation" would mean operationally for credit-card offers. Add BibTeX keys if needed.

11.6 Connections to deep choice models and motivation for the hybrid in Part 2(d)

Deep context-dependent models such as Zhang (2025) address menu dependence by allowing utilities to depend on the entire choice set, thereby relaxing IIA and capturing higher-order substitution patterns.[1] However, these models do not, by default, resolve endogeneity. If offer terms are chosen using information correlated with ξ_{jt}, a flexible deep model can still produce biased demand responses because it may fit endogeneity-driven correlations rather than isolate causal sensitivity to p_{jt}. This is particularly relevant when the deep model has enough capacity to absorb latent targeting effects that are not explicitly recorded in the data. Lu(25)[2] is largely agnostic to the functional form of systematic utility, and its sparsity assumption concerns the unobserved component. This suggests a natural hybrid: use a context-dependent architecture for systematic utility while adding a baseline-plus-sparse layer for unobservables as in (21). Part 2(d) implements and validates this idea in a controlled simulation. A simulation is the appropriate validation device because it allows the analyst to inject known sparse shocks, verify that the model recovers them, and separate improvements in fit due to correcting unobservables from improvements due to flexible context modeling. For completeness, it is also useful to situate DeepHalo relative to other nonparametric or kernel-based approaches to context dependence, such as the reproducing-kernel Hilbert space (RKHS) choice model of Yang et al. (2025).[3] RKHS-based methods can offer a different bias–variance tradeoff than deep architectures and can provide alternative regularization pathways, but they do not directly resolve endogeneity unless combined with an explicit strategy for ξ_{jt} (IV, control functions, proxy variables, low-rank structure, or sparse-shock modeling).

11.7 Summary of assumptions and what to look for empirically

Across the approaches discussed above, the critical question is which assumptions are credible in the data environment. IV-based methods require relevance and exclusion. Control-function methods require a price/offer-term equation that produces a valid control for endogeneity. Proxy-variable approaches require that observed proxies are sufficiently informative about ξ_{jt} and satisfy conditional independence assumptions. Factor models require that ξ_{jt} is well approximated by low-rank structure. Lu & Shimizu (2025) require that a sparse approximation to market–product shocks is reasonable and that posterior computation (or regularized optimization) is stable. In the merchant-offer setting, sparsity is plausible if the dominant unobserved shocks are episodic and localized, but this must be supported empirically using diagnostics, sensitivity to prior or penalty scales, and stability across random seeds. These diagnostics are emphasized in our Part 2(b) replication, where we implement the Lu(25) shrinkage component using TensorFlow Probability MCMC (`tfp.mcmc`) and compare its behavior to BLP benchmarks under the simulation designs of Lu & Shimizu (2025).

12 Errors and Unclear Aspects in Lu & Shimizu (2025)

This section records the parts of Lu & Shimizu (2025) that were unclear to implement, underspecified for an end-to-end replication, or required additional design decisions in order to run the Section 4 simulation study in a modern TensorFlow/TensorFlow Probability pipeline. The intent is not to dispute the core contribution, but to make transparent which details materially affect practical replication and where different reasonable implementations can produce different finite-sample numbers. The discussion focuses on items that directly affect the benchmark BLP+IV estimators and the Bayesian shrinkage estimator in Section 4.[2]

12.1 Ambiguities in the spike-and-slab specification and how it maps to an implementable sampler

A first point that can confuse readers is that the paper's spike-and-slab prior is described in prose and equations, but the mapping between the two variance parameters and the labels "spike" versus "slab" is not always reiterated at the point where the prior enters the algorithm. In the paper's mixture-of-normals approximation, the deviation term is modeled as

$$\eta_{jt} \sim (1-\gamma_{jt})\mathcal{N}(0,\tau_0^2) + \gamma_{jt}\mathcal{N}(0,\tau_1^2), \tag{22}$$

with $0 < \tau_0^2 \ll \tau_1^2$, and where $\gamma_{jt} = 0$ corresponds to the spike component and $\gamma_{jt} = 1$ corresponds to the slab component.[2] The paper later recommends $(\tau_0^2, \tau_1^2) = (10^{-3}, 1)$ and notes potential computational issues if τ_1^2/τ_0^2 is too large, motivating an upper bound on this ratio.[2] While this is standard in the shrinkage literature, an explicit sentence near the algorithm box that "τ_0^2 is the spike variance and τ_1^2 is the slab variance" would reduce the risk of implementation mistakes, especially for readers translating the method into code. A second ambiguity is whether the paper expects γ_{jt} to be sampled explicitly as a discrete latent variable in the main MCMC loop, or whether it is acceptable to integrate γ_{jt} out and operate with a collapsed mixture likelihood. The paper's Appendix describes conditional posteriors for η_{jt} that explicitly depend on γ_{jt} (and introduces market-level inclusion probabilities ϕ_t), which is consistent with a Gibbs-style structure in which one alternates between draws of γ_{jt} and draws of continuous parameters.[2] However, the paper's implementation discussion also emphasizes computational convenience and indicates that mixture-of-normals formulations are used because they make computation simpler.[2] From a replication perspective this matters because TensorFlow Probability does not provide a turnkey Gibbs sampler for discrete inclusion indicators in high-dimensional settings, and some "exact" Gibbs updates would require custom kernels (or alternative samplers). This is not a flaw of the paper, but it is a practical underspecification: the paper provides enough information to implement a Gibbs or Metropolis-within-Gibbs sampler in principle, but it does not provide pseudo-code that is directly translatable into a `tfp.mcmc` implementation without additional design choices (kernel selection, parameter blocking, tuning strategy, and how to handle discrete states). A third issue is that the paper's prior structure includes market-specific sparsity parameters ϕ_t (to allow sparsity to vary by market) and states a Beta prior for ϕ_t.[2] The paper's narrative for the Section 4 simulation study states that the prior probability that each market-product deviation is active is 0.5, which corresponds to a symmetric Beta prior; in the text, this is implemented by setting $(a_\phi, b_\phi) = (1, 1)$.[2] In practice, the distinction between a market-specific ϕ_t and a single global ϕ can materially change posterior behavior, because market-specific ϕ_t allows the chain to adaptively treat some markets as dense and others as sparse. If a replication instead uses a single global inclusion weight, it is a modeling simplification that should be stated explicitly (and it can change the interpretation of "Prob." in the simulation tables). This is one of those places where the paper's modeling intent is clear, but the computational implications (and the necessity of faithful market-specific modeling in code) are easy to underestimate.

12.2 Unclear normalization choices that matter for the IV benchmarks and for comparability of reported quantities

The Section 4 tables report an "Int" column, which the paper defines as the market-specific intercept term $\bar{\xi}_t$ in their decomposition of shocks (and uses "Int=the intercept term $\bar{\xi}$" as a table note).[2] In a BLP+IV implementation, intercept handling is not innocuous because including a constant in X and/or Z changes the meaning of the residual $\hat{\xi}$ and can mechanically violate moment conditions. Specifically, if instruments include a constant while ξ_{jt} has a nonzero mean, then the exclusion restriction $\mathbb{E}[Z\xi] = 0$ cannot hold because $\mathbb{E}[\xi] \neq 0$ implies $\mathbb{E}[\mathbf{1} \cdot \xi] \neq 0$. This creates an implementation tension: one can include a constant in X to absorb the mean of ξ, but then "Int" is no longer a free object comparable to $\bar{\xi}$ as reported in the paper, and one must be careful about how $\hat{\xi}$ is constructed and summarized. The paper's tables and notes strongly suggest a convention that avoids this conflict, but the manuscript does not always spell out regressor and instrument columns in a way that is instantly implementable without interpretation. For example, Table 1 and Table 2 present results by listing $(\text{Int}, \beta_p, \beta_w, \sigma, \xi)$, and the accompanying note explains that the "Prob." column is based on posterior probabilities for γ_{jt} under signal

and noise cases.[2] To match these reported objects in code, a replication must make normalization and intercept decisions consistent across the BLP estimators and the shrinkage estimator; otherwise, one risks reporting "Int" and ξ errors that are not comparable across methods even if the underlying demand model is the same. A related normalization issue is within-market centering of cost shifters used as regressors or instruments. The paper's simulation design often uses cost shifters and their transformations as instruments, and it is natural to center them within market to separate market-level mean shifts from within-market variation. However, centering regressors changes the decomposition of residuals: if $w_{c,tj} = w_{tj} - \bar{w}_t$ enters X, then the raw residual $\delta_{tj} - [p_{tj}, w_{c,tj}]'\hat{\beta}$ differs from the residual under uncentered w_{tj} by a market-level shift proportional to $\hat{\beta}_w \bar{w}_t$. A faithful replication therefore has to either (i) avoid centering, or (ii) explicitly correct the residual when reporting $\hat{\xi}$ and "Int." The paper's presentation does not emphasize this accounting step, but it becomes unavoidable once one fixes a concrete X and Z design for the IV benchmarks and wants the reported "Int" to reflect $\bar{\xi}$ rather than an artifact of centering.

12.3 Posterior computation is described, but the exact computational pathway is not turnkey replicable in TensorFlow Probability

The paper includes a detailed posterior inference section and an appendix describing a tailored Metropolis–Hastings strategy. In particular, it describes updating parameter blocks using a "tailored" proposal centered at the conditional posterior mode with covariance derived from the negative Hessian, with scaling $\kappa_\theta = 2.38\,\dim(\theta)^{-1/2}$ and acceptance-rate calibration.[2] It also notes that conditional on some blocks, updating of $\{\bar{\xi}_t\}$ and $\{\eta_{jt}\}$ can be done independently by market, which is an important computational detail in high dimensions.[2] This is a coherent computational plan, but it is not immediately "drop-in" in `tfp.mcmc` for three reasons. First, implementing a tailored MH kernel as described requires repeated computation of gradients and Hessians of the log posterior for each block, and it requires a robust Newton solver for each conditional posterior mode. The paper states concavity properties that facilitate Newton convergence under Gumbel errors and suggests that convergence is fast.[2] Nevertheless, the practical burden of coding a stable Hessian-based mode-finding procedure that runs inside an outer MCMC loop is substantial, especially when one also needs to run a σ search and demand inversion steps. Second, the paper's blocking structure and independence assumptions across proposal blocks are design choices that affect mixing. For example, proposing β independently of $(\bar{\xi}, \eta, r)$ avoids computing cross-derivatives, but it can also slow mixing if there is strong posterior dependence between these blocks.[2] The paper indicates that the random-walk MH works efficiently for some parameters based on their experience.[2] A replicator still needs to decide which blocks should use tailored MH, which can use random-walk MH, what proposal scales to use, and how to tune these scales in a reproducible way. Third, the full model involves high-dimensional latent objects (market-product shocks and potentially random coefficient parameters) whose dimension grows with $T \cdot J$. The paper acknowledges the dimensionality and motivates blocking and within-market independence to improve speed.[2] In practice, these details determine runtime by orders of magnitude, and they are exactly the details a replication must fix to run in reasonable time. Because JPM explicitly requested using `tfp.mcmc` and acknowledged that TFP does not implement Gibbs, the practical constraint is to choose an MCMC procedure that is both faithful to the modeling intent and feasible to implement and run. This creates a legitimate gap between "paper algorithm" and "TFP implementable algorithm." The paper does not resolve this gap because it is not written with TFP constraints in mind, but a replication must resolve it and document the consequences.

12.4 Finite-sample guidance and diagnostics are limited relative to what an applied user needs

The paper's identification arguments are asymptotic and rely on conditions (Assumptions 1–3 and Lemma-based inversion steps) that ensure identification of the taste distribution and shocks under sparsity.[2] The theoretical logic is sound, but an applied reader implementing the model will still need operational guidance on when the sparse-shock restriction is a good approximation, how to detect failure modes (e.g., when the posterior concentrates on dense patterns or when shrinkage absorbs variation that should be attributed to price), and what chain lengths and tuning are required for stable inference at the simulation sizes used in Section 4. This is particularly important because the Section 4 pipeline is computationally nested: one evaluates candidate σ values and, for each σ, performs share inversion (with simulation draws) and then runs an MCMC routine for shrinkage. The paper does not provide a runtime scaling discussion or default tuning rules that would let practitioners budget compute. This omission is understandable in an econometrics paper, but it becomes practically relevant in replication work, because differences in chain length, burn-in, thinning, and proposal scale can translate directly into differences in the reported "Bias/SD" entries of the Section 4 tables.

12.5 Reproducibility and randomness control in TensorFlow-based replications

Finally, some "unclear parts" are not about the econometric model but about reproducibility in modern ML tooling. Both demand inversion (through Monte Carlo integration over taste draws) and the MCMC routines depend on random number generation. TensorFlow's stateful RNG and hardware-dependent kernels can produce run-to-run variation if not carefully controlled. This matters because the Section 4 outputs are Monte Carlo averages over replications, and small nondeterminism at the inner-loop level can be amplified when σ is chosen by comparing noisy objective values across a grid. A robust replication therefore needs to (i) seed simulation data generation per replication, (ii) seed the taste-draw simulation used in share inversion per replication, and (iii) seed the MCMC routine per replication. Even with careful seeding, exact bitwise reproducibility across CPU versus GPU/Metal backends is not always guaranteed in TensorFlow. The paper does not discuss these engineering details (which is reasonable), but in a TensorFlow/TFP replication they become part of the required "missing specification" that determines whether results are stable.

12.6 Summary: what is genuinely unclear versus what is a deliberate implementation choice

The core modeling idea in Lu & Shimizu (2025) is clear: impose sparsity structure on market–product shocks and estimate them directly rather than relying exclusively on IVs.[2] The places where a replicator must fill in gaps are primarily computational and normalization-related. The most consequential are the exact posterior sampling strategy (especially under `tfp.mcmc` constraints), the treatment of intercepts and centering so that "Int" and ξ error metrics match the paper's definitions, and the tuning and runtime tradeoffs inherent in a nested σ search with MCMC at each grid point. In Part 2(b), we make our choices explicit, describe how they differ from the paper's suggested tailored MH framework, and use these differences to motivate hypotheses for any remaining discrepancies between our numbers and the paper's reported simulation tables.

13 Replicating Lu & Shimizu (2025), Section 4 Simulation Study

13.1 Goal and scope of the replication

Lu & Shimizu (2025) study the identification and estimation of discrete choice demand models when market–product demand shocks (unobserved market–product characteristics) are present and may be correlated with price-like variables.[2] Their Section 4 simulation study compares classical BLP-style IV benchmarks to a shrinkage estimator that treats the unobserved shocks as structured objects and estimates them under a sparsity-inducing model. The goal of this subsection is an *approximate reimplementation* of the Section 4 simulation study: we reproduce the DGP structure and qualitative patterns, but make no claim of exact numerical replication. Key deviations are documented in Section 13.5.

We implemented a canonical replication driver `jpm_q3.lu25.experiments.replicate_section4` (invoked via `jpmq3-replicate-lu25`). The driver runs the standard grid point $(T, J) = (25, 15)$ for DGP1–DGP4 and produces paper-style Bias/SD tables for Int, β_p, β_w, σ, and shock recovery metrics for ξ. The shrinkage component is implemented with TensorFlow Probability MCMC (`tfp.mcmc`) as requested by JPM; because `tfp.mcmc` does not provide a Gibbs sampler for discrete inclusion indicators, our MCMC differs from a textbook Gibbs sampler and we state these deviations explicitly below.

13.2 Benchmark demand system and share inversion

Throughout Section 4, the simulation generates markets indexed by $t \in \{1, \dots, T\}$ with inside goods $j \in \{1, \dots, J\}$ and an outside option $j = 0$. For a given candidate random-coefficient parameter σ, the BLP-style object that is inverted from shares is the mean utility vector $\delta_t(\sigma) \in \mathbb{R}^J$ for each market t. In our implementation, $\delta(\sigma)$ is computed by the standard BLP contraction mapping using a finite number R of simulation draws for heterogeneity:

$$\delta_t^{(m+1)} \;=\; \delta_t^{(m)} + \log s_t^{\text{obs}} - \log s_t\Big(\delta_t^{(m)}; \sigma\Big)\,, \tag{23}$$

where $s_t(\cdot; \sigma)$ is the model-implied share mapping at heterogeneity parameter σ and the logarithm is applied elementwise on inside-good shares. The finite-R approximation enters through the simulation approximation of $s_t(\cdot; \sigma)$, so larger R reduces Monte Carlo noise in the inverted $\delta(\sigma)$.

13.3 BLP benchmark estimators (with and without cost instruments)

Mean-utility regression and residual shocks. After computing $\delta(\sigma)$, the IV benchmarks estimate a linear mean-utility regression of the form

$$\delta_{tj}(\sigma) \;=\; X_{tj}'\beta + \xi_{tj}, \tag{24}$$

where X_{tj} contains observed regressors and ξ_{tj} is the unobserved market–product shock. In our notation, $\beta = (\beta_p, \beta_w)$ corresponds to the coefficients on the price-like regressor and the cost-shifter regressor, respectively. For a given σ, we estimate $\beta(\sigma)$ by 2SLS using instruments Z, and then form residual shocks

$$\hat{\xi}^{\text{raw}}(\sigma) \;=\; \delta(\sigma) - X\hat{\beta}(\sigma). \tag{25}$$

Paper-aligned regressor and instrument matrices (no constant). A practical replication issue in Section 4 is that the paper reports an intercept-like term "Int" corresponding to the mean of ξ (denoted in the paper as $\bar{\xi}$), while simultaneously relying on IV moments that would be violated if a constant instrument were included when $\bar{\xi} \neq 0$.[2] To keep the IV moments meaningful and to keep "Int" interpretable as the mean of the recovered ξ, our replication follows a paper-aligned convention: we exclude the constant from both X and Z and use a within-market centered cost shifter

$$w_{c,tj} \;=\; w_{tj} - \overline{w}_t, \qquad \overline{w}_t \;=\; \frac{1}{J}\sum_{j=1}^{J} w_{tj}. \tag{26}$$

We then set

$$X \;=\; [p_{tj},\, w_{c,tj}], \tag{27}$$

and use two instrument sets:

$$Z_{\text{cost}} \;=\; [w_{c,tj},\, w_{c,tj}^2,\, u_{tj},\, u_{tj}^2], \qquad Z_{\text{nocost}} \;=\; [w_{c,tj},\, w_{c,tj}^2,\, w_{c,tj}^3,\, w_{c,tj}^4]. \tag{28}$$

These match the intended distinction in the paper between a cost-IV design and a design that omits cost information and relies only on transformations of w.[2]

Centering correction for ξ and the reported "Int". Because w is centered in X, the raw residual $\hat{\xi}^{\text{raw}}$ differs from the residual one would obtain if the uncentered w were used, by a market-level shift proportional to $\hat{\beta}_w \overline{w}_t$. To ensure that the reported shock aligns with the paper's decomposition $\xi = \bar{\xi} + \eta$ and that the reported "Int" corresponds to $\bar{\xi}$ rather than to a centering artifact, we apply a deterministic correction:

$$\hat{\xi}_{tj} \;=\; \hat{\xi}^{\text{raw}}_{tj} - \hat{\beta}_w \, \overline{w}_t. \tag{29}$$

We then report

$$\widehat{\text{Int}} \;=\; \frac{1}{TJ} \sum_{t=1}^{T} \sum_{j=1}^{J} \hat{\xi}_{tj}. \tag{30}$$

This correction is applied identically for both BLP+IV benchmarks and for the shrinkage estimator so that the "Int" and ξ metrics are comparable across methods.

GMM objective and selection of $\hat{\sigma}$ (BLP). For each candidate σ, we form IV residuals $\hat{\xi}(\sigma)$ and compute the sample moments

$$g(\sigma) \;=\; \frac{1}{N} Z^\top \hat{\xi}(\sigma), \qquad N = TJ. \tag{31}$$

We use a one-step weighting matrix

$$W \;=\; \left(\frac{1}{N} Z^\top Z \right)^{-1}, \tag{32}$$

and define the GMM objective

$$Q(\sigma) \;=\; g(\sigma)^\top W g(\sigma). \tag{33}$$

We select $\hat{\sigma}$ using a coarse grid over $\sigma \in [0.05, 4.0]$ followed by a local refinement around the best coarse value (grid+refine). This is a stable and transparent approach for replication, even though it may not match every optimization heuristic used in the paper.

13.4 Shrinkage estimator implemented with `tfp.mcmc`

Shrinkage problem conditional on $\delta(\sigma)$. The shrinkage estimator treats the unobserved shock as a structured object rather than as a residual to be eliminated by IV. Conditional on $\delta(\sigma)$ and X, we write

$$\delta \;=\; X\beta + \xi, \tag{34}$$

and impose a spike-and-slab-inspired structure on ξ. In the paper, sparsity is expressed through inclusion indicators and a mixture prior; this is the key modeling device that enables recovery of shocks even in settings where instruments are weak or exclusion is fragile.[2]

Collapsed spike-and-slab likelihood used in our TFP implementation. To satisfy the constraint that the shrinkage component must be implemented using `tfp.mcmc`, and given that TFP does not provide a Gibbs sampler for discrete inclusion variables, we use a collapsed formulation that integrates out the discrete inclusion indicator for each coordinate. We introduce a global inclusion probability π with Beta prior

$$\pi \sim \text{Beta}(a_\pi, b_\pi), \tag{35}$$

and a two-component normal mixture for residuals. Writing residuals $r_n = \delta_n - x_n^\top \beta$ (where n indexes stacked (t, j)), the collapsed likelihood is

$$p(r_n \mid \beta, \pi) \;=\; (1-\pi)\,\mathcal{N}(r_n; 0, v_0) \;+\; \pi\,\mathcal{N}(r_n; 0, v_1), \tag{36}$$

where v_0 is the spike variance and v_1 is the slab variance. This preserves the key modeling intent (a sparse-versus-non-sparse mixture) while avoiding explicit sampling of γ_n.

TFP MCMC kernel and target density. We sample the continuous parameters $(\beta, \text{logit}(\pi))$ using `tfp.mcmc.RandomWalkMetr`
The target log density is

$$\log p(\beta, \pi \mid \delta, X) = \sum_{n=1}^{N} \log\Big((1-\pi)\mathcal{N}(r_n; 0, v_0) + \pi \mathcal{N}(r_n; 0, v_1) \Big) + \log p(\beta) + \log p(\pi) + \log \left| \frac{d\pi}{d\,\text{logit}(\pi)} \right|, \tag{37}$$

where we use a diffuse Gaussian prior $p(\beta) = \mathcal{N}(0, \beta_{\text{var}} I)$ and include the Jacobian term for the logit transform. Each replication uses a seed derived from the replication seed so that the MCMC trajectory is reproducible conditional on the dataset seed.

Selecting $\hat{\sigma}$ under shrinkage. We evaluate candidate σ values using the same grid+refine scheme. For each σ, we run the MCMC chain and compute a scalar "score" equal to the mean sampled log posterior after burn-in and thinning. We then choose

$$\hat{\sigma} \;=\; \arg\max_{\sigma} \; \text{score}(\sigma). \tag{38}$$

At $\hat{\sigma}$, we compute $\hat{\xi}$ using the residual $\delta(\hat{\sigma}) - X\hat{\beta}$ and apply the same centering correction in (29) so that the reported $\hat{\xi}$ is comparable to the BLP residual-based $\hat{\xi}$.

Posterior inclusion probabilities used for the "Prob." column. Although we do not explicitly sample inclusion indicators, the collapsed mixture yields an analytic posterior inclusion probability for each coordinate conditional on (β, π):

$$\Pr(\gamma_n = 1 \mid r_n, \pi) = \frac{\pi\,\mathcal{N}(r_n; 0, v_1)}{(1-\pi)\mathcal{N}(r_n; 0, v_0) + \pi\,\mathcal{N}(r_n; 0, v_1)}. \tag{39}$$

We compute these probabilities and report their averages on signal versus noise coordinates when the simulator provides a ground-truth signal mask.

13.5 Implementation summary and explicit differences versus the paper

The replication driver writes, per DGP cell, `summary.csv` (long-format metrics), `paper_table_like.csv` (Bias/SD table), and `config.json` (true parameters and run metadata). The driver also tracks method-level failures and reports a fail rate, enabling diagnosis of numerical instabilities. The key ways our implementation differs from the paper are direct consequences of the `tfp.mcmc` constraint and of making the regression/instrument design explicit. First, we use `tfp.mcmc.RandomWalkMetropolis` rather than the paper's tailored Metropolis–Hastings design with Hessian-based proposals, because implementing the tailored kernel inside `tfp.mcmc` would require substantial custom code and repeated Hessian computations, and JPM explicitly prioritized use of `tfp.mcmc` over exact numerical replication. Second, we use a collapsed mixture likelihood and do not explicitly sample discrete inclusion indicators. This changes the Markov chain's state space relative to a Gibbs-style sampler, so exact finite-sample matches are not expected even when the modeling assumptions align. Third, we select σ via a transparent grid+refine approach for both BLP and shrinkage; the paper may use different optimization heuristics. Fourth, we invert shares using a finite number of simulation draws R, which introduces approximation noise that can affect $\hat{\sigma}$ in finite samples. Finally, we make intercept and centering choices explicit by excluding constants from X and Z and applying a correction to $\hat{\xi}$; this is necessary to keep IV moments coherent and to make "Int" comparable across methods.

13.6 Run configuration

For the main replication table reported here, we run DGP1–DGP4 at $(T, J) = (25, 15)$ with

$$\texttt{n\_reps} = 10, \qquad R = 50, \qquad \texttt{n\_jobs} = 4, \tag{40}$$

and use the simulator's true parameters (as written in `config.json`):

$$\beta_p^\star = -1.0, \quad \beta_w^\star = 0.5, \quad \sigma^\star = 1.5, \quad \text{Int}^\star = -1.0. \tag{41}$$

These settings are sufficient to reproduce the qualitative patterns in Section 4, but they are not identical to the largest-grid settings sometimes used in published simulation tables; the finite-R and finite-`n_reps` choices are therefore a plausible contributor to remaining numeric differences.

13.7 Replication outputs (paper-style table)

13.8 Known limitation: BLP $\hat{\sigma}$ bias from small R

The BLP+CostIV σ bias is large and negative across all four DGPs (≈ -1.25 against true $\sigma^\star = 1.5$, a relative bias of -83%). This is a known finite-R artefact of the BLP contraction mapping: with only $R = 50$ simulation draws for heterogeneity, the Monte Carlo noise in the simulated share mapping $s_t(\delta; \sigma)$ flattens and distorts the GMM objective surface $Q(\sigma)$, causing the grid+refine search to select $\hat{\sigma}$ well below the truth. The paper's own implementation uses a substantially larger R; the discrepancy here is entirely attributable to $R = 50$, not to a modelling error. The qualitative comparison between BLP and shrinkage remains valid because both BLP variants are subject to the same finite-R noise. We state this limitation explicitly so the reader does not interpret the BLP σ bias as evidence that the BLP estimator is structurally broken.

Table 2: Simulation results (our replication) for DGP1–DGP4 at $(T, J) = (25, 15)$ in the paper-style layout. For each DGP and each estimator, we report Bias and SD across Monte Carlo replications for Int $= \frac{1}{TJ}\sum_{t,j}\hat{\xi}_{tj}$, β_p, β_w, and σ. Shock recovery is summarized by MAE($\hat{\xi} - \xi^{\text{true}}$) and SD($\hat{\xi} - \xi^{\text{true}}$). For the Shrinkage estimator, "Prob(signal)" and "Prob(noise)" report mean posterior inclusion probabilities over true signal vs. true noise coordinates when the simulator provides that mask; when noise labels are unavailable or empty, we report "–".

DGP	Method	Row	Int	β_p	β_w	σ	MAE($\hat{\xi} - \xi^{\text{true}}$)	SD($\hat{\xi} - \xi^{\text{true}}$)	Prob(signal)	Prob(noise)
DGP1 (sparse exogenous)										
DGP1	BLP+CostIV	Bias	+0.226	+0.387	-0.063	-1.247	0.350	0.372	–	–
DGP1	BLP+CostIV	SD	0.229	0.104	0.143	0.341			–	–
DGP1	BLP-NoCostIV	Bias	+0.302	-0.257	-0.035	-0.631	0.508	0.557	–	–
DGP1	BLP-NoCostIV	SD	0.227	1.037	0.148	1.782			–	–
DGP1	Shrinkage	Bias	+0.077	-0.180	+0.023	-0.015	0.141	0.178	0.986	0.346
DGP1	Shrinkage	SD	0.075	0.069	0.050	0.137			–	–
DGP2 (sparse endogenous)										
DGP2	BLP+CostIV	Bias	+0.230	+0.348	-0.055	-1.362	0.360	0.396	–	–
DGP2	BLP+CostIV	SD	0.209	0.065	0.134	0.199			–	–
DGP2	BLP-NoCostIV	Bias	+0.303	-0.482	-0.032	-0.218	0.569	0.640	–	–
DGP2	BLP-NoCostIV	SD	0.209	1.110	0.138	2.048			–	–
DGP2	Shrinkage	Bias	+0.071	-0.162	+0.016	-0.009	0.131	0.160	0.992	0.351
DGP2	Shrinkage	SD	0.082	0.050	0.055	0.110			–	–
DGP3 (approximately sparse)										
DGP3	BLP+CostIV	Bias	+0.133	+0.403	+0.006	-1.317	0.273	0.360	–	–
DGP3	BLP+CostIV	SD	0.125	0.130	0.079	0.350			–	–
DGP3	BLP-NoCostIV	Bias	+0.196	-0.503	+0.031	-0.218	0.480	0.600	–	–
DGP3	BLP-NoCostIV	SD	0.151	1.217	0.105	2.048			–	–
DGP3	Shrinkage	Bias	-0.039	-0.127	+0.096	-0.051	0.172	0.168	0.302	–
DGP3	Shrinkage	SD	0.176	0.047	0.120	0.106			–	–
DGP4 (approximately sparse)										
DGP4	BLP+CostIV	Bias	+0.118	+0.363	+0.016	-1.252	0.274	0.357	–	–
DGP4	BLP+CostIV	SD	0.150	0.154	0.101	0.421			–	–
DGP4	BLP-NoCostIV	Bias	+0.211	-0.497	+0.019	-0.218	0.503	0.611	–	–
DGP4	BLP-NoCostIV	SD	0.230	1.199	0.156	2.048			–	–
DGP4	Shrinkage	Bias	-0.069	-0.062	+0.100	-0.079	0.174	0.155	0.340	–
DGP4	Shrinkage	SD	0.194	0.060	0.131	0.115			–	–

13.9 Key qualitative findings

The most robust qualitative finding across all four DGPs is that the shrinkage estimator substantially improves shock recovery relative to the IV benchmarks. In every DGP, the shrinkage estimator produces markedly smaller ξ mean absolute error than both BLP+CostIV and BLP-NoCostIV, which is consistent with the paper's central claim that directly modeling sparse shocks can recover unobservables better than treating shocks purely as residuals after IV regression. In addition, the BLP-NoCostIV benchmark is numerically unstable in the sense that it exhibits very large sampling variability for β_p and σ, which is consistent with weak identification when cost instruments are removed and the remaining polynomial instruments provide insufficient independent variation. A second robust qualitative pattern is that the shrinkage estimator produces relatively stable σ estimates in these runs, whereas the BLP+CostIV benchmark exhibits a large negative bias in σ. The stability of shrinkage is not surprising in itself, because the shrinkage model is allowed to absorb a substantial portion of residual variation into the shock distribution; however, the magnitude and direction of the BLP σ bias is a clear place where our finite-sample outputs differ from what one might expect from a well-tuned BLP replication, and it therefore motivates a set of concrete hypotheses.

13.10 Where our numbers differ from the paper and hypotheses

The largest discrepancy in our outputs is the large negative bias in σ for BLP+CostIV across all DGPs. A first hypothesis is that finite-R simulation error in share inversion materially affects the shape of the GMM objective $Q(\sigma)$, because $\delta(\sigma)$ is computed using simulated shares. When R is small (we used $R = 50$ in these runs), the Monte Carlo noise in $\delta(\sigma)$ can distort the objective surface enough that a coarse grid+refine search selects a systematically different $\hat{\sigma}$ than would be obtained with a larger R. A second hypothesis is that the grid+refine search itself can miss a narrow optimum if the objective is relatively flat or multi-modal at the grid resolution; this is an optimization discrepancy rather than a modeling discrepancy. A third hypothesis is that the one-step weighting matrix $W = (Z^\top Z/N)^{-1}$ interacts with instrument conditioning; if Z is ill-conditioned (especially in polynomial instrument designs), then small changes in $\hat{\xi}(\sigma)$ can create large swings in $Q(\sigma)$, again affecting the

selected $\hat{\sigma}$. Finally, differences between our shrinkage MCMC kernel and the paper's tailored MH design can shift $\hat{\sigma}$ under shrinkage as well; however, JPM explicitly prioritized implementing the shrinkage component using `tfp.mcmc` over exact numerical matching, so the relevant standard is that the MCMC-based shrinkage pipeline is correct and reproducible, not that it exactly matches the paper's finite-sample numbers.

13.11 Diagnostics and reproducibility

All DGP cells completed with a reported fail rate of 0% for all three methods, which indicates that the replication pipeline is operational end-to-end and that numerical issues are not being hidden by silent failures. Each Monte Carlo replication uses a unique dataset seed, and the shrinkage MCMC uses a per-replication seed derived from the replication seed, which makes reruns deterministic conditional on the same run configuration. The shrinkage routine reports an acceptance-rate diagnostic from `tfp.mcmc.RandomWalkMetropolis`; in our completed runs the procedure did not exhibit degenerate behavior (such as acceptance collapsing near 0 or exploding near 1) and all cells completed without MCMC-related failures. Finally, all saved artifacts (`paper_table_like.csv`, `summary.csv`, and `config.json`) are sufficient to regenerate Table **??** directly from the run folders.

14 Benchmark Models and the Instrumental Variables Approach

Lu & Shimizu (2025) position their sparse-shock estimator relative to classical approaches that handle price endogeneity using instrumental variables within a BLP-style random-coefficient logit demand system.[2] This section explains the benchmark models used in the simulation study, the assumptions required for BLP+IV to consistently estimate price sensitivity, the types of variables that are typically observed versus unobserved in choice-model data, and what constitutes a plausible instrument in a credit card merchant-offer setting. We also describe an additional benchmark class (control function) that can be used as a robustness check.

14.1 BLP demand model and where endogeneity enters

In the BLP framework, consumer i in market t obtains utility from product j according to

$$u_{ijt} \;=\; x_{jt}'\beta_i \;-\; \alpha_i p_{jt} \;+\; \xi_{jt} \;+\; \varepsilon_{ijt}, \tag{42}$$

where x_{jt} are observed product characteristics, p_{jt} is a price-like variable, ξ_{jt} is an unobserved market–product demand shock, and ε_{ijt} is an idiosyncratic taste shock typically assumed to be i.i.d. Type-I extreme value. Heterogeneity is captured through a mixing distribution for (β_i, α_i), which induces flexible substitution patterns.

The econometric problem is that firms or designers of offers typically observe components of ξ_{jt} when setting p_{jt} or choosing offer terms. In a credit-card offer context, p_{jt} may represent APR/fees, required spend thresholds, or an effective "price" defined as the negative of reward generosity, while ξ_{jt} can represent unobserved targeting intensity, channel exposure decisions, co-funding frictions, or latent merchant appeal. If the offer designer chooses p_{jt} using information correlated with ξ_{jt}, then $\mathrm{Cov}(p_{jt}, \xi_{jt}) \neq 0$ and the price coefficient is not identified by naive regression of mean utilities on observables.

BLP separates computation from identification. For a given random-coefficient parameter σ, observed market shares are inverted to recover mean utilities $\delta_{jt}(\sigma)$, and then the linear relationship

$$\delta_{jt}(\sigma) \;=\; x_{jt}'\beta \;-\; \alpha p_{jt} \;+\; \xi_{jt} \tag{43}$$

is estimated using GMM moment conditions that instrument for the endogenous component of p_{jt}.

14.2 Benchmark estimators and how they relate to our replication

Lu & Shimizu (2025) report benchmark estimators based on BLP with two different instrument sets and compare them to their shrinkage estimator.[2] In our replication of Section 4, we implemented the two BLP benchmarks and the shrinkage estimator, and we aligned the regressor and instrument definitions to the paper's treatment of the nonzero mean shock ("Int") by excluding constants from both X and Z and using within-market centering of the cost shifter.

After share inversion, the BLP benchmarks estimate a mean-utility regression of the form

$$\delta(\sigma) \;=\; X\beta + \xi, \tag{44}$$

and use instruments Z to enforce orthogonality $\mathbb{E}[Z'\xi] = 0$. In our replication, the regressor matrix excludes a constant and is defined as

$$X \;=\; [p_{tj},\; w_{c,tj}], \qquad w_{c,tj} = w_{tj} - \overline{w}_t, \tag{45}$$

where $\overline{w}_t$ is the within-market mean of w_{tj} across inside goods. We used two instrument sets matching the intended distinction in the paper:

$$Z_{\text{cost}} \;=\; [w_{c,tj},\; w_{c,tj}^2,\; u_{tj},\; u_{tj}^2], \qquad Z_{\text{nocost}} \;=\; [w_{c,tj},\; w_{c,tj}^2,\; w_{c,tj}^3,\; w_{c,tj}^4], \tag{46}$$

where u_{tj} is a cost shock (available in simulation) and the "NoCostIV" benchmark removes u_{tj} and relies only on polynomial transformations of $w_{c,tj}$. This benchmark corresponds to the empirically common situation where truly exogenous cost-side instruments are unavailable and one must rely on weaker proxies.

We emphasize that the paper and our replication report an intercept-like quantity "Int" corresponding to the mean of ξ. When $\bar{\xi} \neq 0$, including a constant instrument would mechanically violate $\mathbb{E}[Z\xi] = 0$. This is why our implementation excludes constants from X and Z and applies a deterministic correction to the residual shock so that the reported $\hat{\xi}$ corresponds to the paper's decomposition $\xi = \bar{\xi} + \eta$ (details are in Part 2(b)).

14.3 Assumptions required for BLP+IV identification under price endogeneity

BLP+IV relies on the same core identifying assumptions as linear IV, with additional regularity conditions for the share inversion and the GMM objective.

First, the *exclusion restriction* requires that instruments do not directly enter utility once x_{jt} is controlled for:

$$\mathbb{E}[Z_{jt}\,\xi_{jt}] = 0 \quad \text{for all } j, t. \tag{47}$$

This assumption is fundamentally untestable and must be defended by institutional or economic reasoning. In a demand setting, valid instruments are typically supply-side shifters that affect equilibrium prices through costs or constraints but do not affect consumer utility directly.

Second, *relevance* requires that instruments shift the endogenous price-like variable:

$$\mathrm{Cov}(Z_{jt}, p_{jt}) \neq 0, \tag{48}$$

or more generally that the first-stage projection of p_{jt} on Z_{jt} has sufficient strength. Weak instruments lead to unstable GMM objectives and large sampling variability, which is exactly what the BLP-NoCostIV benchmark is designed to illustrate.

Third, a *rank condition* is required so that the moment system is informative:

$$\mathrm{rank}\big(\mathbb{E}[Z_{jt}Z_{jt}']\big) = \dim(Z_{jt}), \tag{49}$$

and in practice this fails when instruments are nearly collinear (which can occur with polynomial instruments or when product sets lack variation).

Finally, BLP requires correct specification of the utility model (including the distributional assumptions for heterogeneity and ε_{ijt}) and well-behaved share inversion. The inversion step requires that the mapping from mean utilities to shares is invertible and that the contraction mapping converges to a unique fixed point under standard conditions.**berry1994estimating**, **berry1995automobile**

14.4 Observed versus unobserved variables in choice-model data and why IV is used

In typical choice-model datasets, the analyst observes product attributes x_{jt}, a price-like variable p_{jt} (or offer terms that can be mapped to a price concept), market identifiers, and either individual choices or aggregate shares. In the credit-card merchant-offer setting, observed offer attributes include reward amount, spend threshold, eligibility restrictions, merchant category, and timing; observed context variables may include channel, geography, and broad customer segment. However, the analyst often does not observe campaign intensity, targeting rules, internal propensity/uplift scores, vendor co-funding decisions, or operational constraints, all of which can enter utility and therefore constitute ξ_{jt}.

Endogeneity arises because the offer designer observes or forecasts these latent factors and sets offer terms accordingly. As a result, p_{jt} is correlated with ξ_{jt}, and without an additional source of variation (such as valid instruments), the estimated price sensitivity confounds causal price response with selection on unobserved demand conditions.

14.5 How to find instruments and three viable examples in the credit card offer setting

A viable instrument must shift offer terms in a way that is plausibly orthogonal to latent demand shocks after controlling for observed covariates. In the credit-card offer setting, the most defensible instruments are typically cost-side or constraint-based shifters rather than competitor-based variables, because competitive conditions often co-move with latent demand.

A first viable instrument class is a *bank funding-cost shifter* such as a bank-specific cost-of-funds spread or an index of marginal funding rates. Funding costs affect the profitability of extending credit and therefore influence APR/fee policies and the feasible generosity of rewards. Exclusion is plausible because consumers do not directly derive utility from the bank's funding cost, although one must control for macro conditions if funding costs are correlated with demand-side cycles.

A second viable instrument class is *regulatory or network-fee shifters* that move interchange or scheme fees (or change compliance costs) in ways that affect the marginal cost of offers. Changes in network pricing schedules or regulatory fee updates can shift equilibrium offer terms without directly affecting consumer tastes, provided that the timing of the regulatory change is not itself driven by contemporaneous demand shocks for a specific offer.

A third viable instrument class is *merchant-side cost or subsidy shifters* that affect the bank/vendor economics but do not directly enter consumer utility once observed offer attributes are included. Examples include merchant

co-funding budget changes driven by accounting cycles, changes in redemption reimbursement rates, or merchant settlement costs that are contractually determined. These can move offer generosity through vendor economics while remaining plausibly orthogonal to latent demand shocks conditional on observed merchant and timing controls.

We note that competitor-offer characteristics are sometimes used as BLP-style instruments, but in merchant-offer settings they are often difficult to justify because industry-wide seasonality and coordinated promotional periods generate correlated demand shocks across issuers. As a result, competitor-based instruments are best treated as supplementary rather than primary instruments unless one can credibly argue that demand shocks are independent across issuers conditional on controls.

14.6 Another benchmark model: control functions for endogeneity

A natural additional benchmark is a control-function approach, which introduces an explicit price-setting or offer-term equation and uses the residual from that equation as a control in utility. Concretely, one estimates a first-stage model

$$p_{jt} = Z'_{jt}\pi + x'_{jt}\rho + v_{jt}, \tag{50}$$

and then includes $\hat{v}_{jt}$ as an additional regressor in the mean-utility equation so that conditional on $\hat{v}_{jt}$ the remaining variation in p_{jt} is plausibly exogenous. In random-coefficient settings, this can be incorporated into the likelihood or simulated moments as in control-function extensions of BLP.[4]

This benchmark does not eliminate the need for valid instruments, because the first stage still requires exogenous variation in p_{jt}. However, it can provide a useful robustness check, and in some settings it is numerically more stable than GMM with a poorly conditioned weighting matrix.

14.7 Summary

The BLP+IV benchmarks represent the classical identification strategy for price endogeneity: recover mean utilities by share inversion and use exogenous instruments to isolate variation in price that is orthogonal to unobserved demand shocks. The main difficulty in practice is defending exclusion and ensuring instrument strength. Lu & Shimizu (2025) propose an alternative identification strategy that replaces exclusion with a structural sparsity assumption on ξ_{jt}, and their simulation benchmarks are designed to show that IV performance degrades sharply as instruments weaken. In a credit-card offer setting, defensible instruments are most likely to come from bank funding costs, regulatory/network fee changes, or merchant-side cost and subsidy shifters, while competitor-based instruments require substantially stronger assumptions about the independence of unobserved demand shocks across issuers.

15 Integrating Lu(25)-style sparse unobservables into Zhang(25)

Problem formulation. Part 1 focuses on context-dependent substitution patterns but does not explicitly model market–product unobserved demand shocks (unobserved characteristics) that enter utility and can induce endogeneity when they are correlated with offer terms such as price-like variables. Lu & Shimizu (2025) propose handling these unobservables by modeling them directly under a structural sparsity assumption rather than relying only on instrumental variables. Zhang (2025)'s DeepHalo focuses on context effects in utilities but does not impose a specific structure on market–product unobservables. The goal in Question 3(d) is therefore to modify DeepHalo so that it admits Lu-style sparse unobservables and to validate the modification in a controlled simulation where the true shock structure is known.

Model: Zhang-Sparse (DeepHalo + Lu-style shocks). Let $t \in \{1, \dots, T\}$ index markets and $j \in \{0, 1, \dots, J\}$ index products, with $j = 0$ the outside option. DeepHalo produces context-dependent baseline utilities

$$u_{tj}^{\text{halo}} = f_\theta(j, S_t, x_{t,1:J}),$$

where f_θ is the DeepHalo architecture applied to the choice set S_t and item identifiers/features. To incorporate Lu(25)'s sparse unobservables, we augment utilities with a market intercept μ_t and market–product deviations d_{tj} (defined only for inside goods):

$$u_{tj} = u_{tj}^{\text{halo}} + \mathbb{1}\{j \neq 0\}\mu_t + \mathbb{1}\{j \neq 0\}d_{tj}. \tag{51}$$

The outside option is normalized to have zero shock, so μ_t and d_{tj} apply only for $j \neq 0$. Given the total utilities u_{tj}, the multinomial logit choice probabilities are

$$P_{tj} = \frac{\exp(u_{tj})}{\sum_{k \in S_t \cup \{0\}} \exp(u_{tk})}. \tag{52}$$

Identification via within-market centering. Because μ_t and d_{tj} are additive, they are not separately identified without a normalization: for any constant c_t, replacing $\mu_t \leftarrow \mu_t + c_t$ and $d_{tj} \leftarrow d_{tj} - c_t$ leaves $\mu_t + d_{tj}$ unchanged. We therefore impose the within-market centering constraint on inside-good deviations,

$$\frac{1}{J}\sum_{j=1}^{J} d_{tj} = 0 \quad \text{for each } t, \tag{53}$$

which preserves the interpretation of μ_t as a market-level mean shock and d_{tj} as a zero-mean deviation within the market.

Estimation objective (MAP). We estimate (θ, μ, d) by maximum a posteriori (MAP) using a negative log-likelihood term plus an ℓ_1 penalty on d and a Gaussian prior (ridge) on μ:

$$\min_{\theta,\mu,d} \; -\sum_{n=1}^{N} \log P\big(y_n \mid S_{t(n)}, \theta, \mu, d\big) + \lambda \sum_{t=1}^{T}\sum_{j=1}^{J} |d_{tj}| + \frac{1}{2\sigma_\mu^2}\sum_{t=1}^{T} \mu_t^2. \tag{54}$$

The ℓ_1 term encourages many d_{tj} to be exactly or approximately zero, which is the likelihood-based analog of Lu(25)'s sparsity assumption, while the ridge prior stabilizes market intercepts.

Implementation note (no MCMC in this subsection). This Zhang-Sparse extension is implemented via MAP optimization in TensorFlow; we do *not* use MCMC/TFP in this subsection. TensorFlow Probability (`tfp.mcmc`) is used for the Lu(25) shrinkage replication in Part 2(b), where MCMC is explicitly required by the prompt and JPM feedback.

Training algorithm (two-stage). In practice, jointly training (θ, μ, d) can allow the deep context network to absorb variation that should be attributed to sparse shocks, making it harder to diagnose whether sparsity is being recovered. To make the role of the sparse layer explicit and easier to validate, we use a two-stage procedure:

1. **Stage 1 (context fit):** set $\lambda = 0$ and do not train d (equivalently, keep $d \equiv 0$). Train DeepHalo parameters θ and market intercepts μ by minimizing (54) with the ℓ_1 term disabled.
2. **Stage 2 (sparse shocks):** freeze θ and train (μ, d) by minimizing the full MAP objective (54), enforcing the centering constraint (53) in the forward pass.

This design forces d to explain residual market–product structure after context effects have been learned.

Simulation study design (validation). We validate the modification using a simulation in which the ground truth contains both context effects and sparse shocks:

1. Fix T markets and J inside goods (plus outside option). Choose a sparsity rate (we report two regimes, 20% and 40% nonzero entries per market in $d_{t\cdot}$).
2. Generate baseline context utilities $u_{t\cdot}^{\text{halo}}$ by running the same DeepHalo architecture with a fixed initialization.
3. Sample market intercepts $\mu_t \sim \mathcal{N}(0, \sigma_\mu^2)$ and sparse deviations d_{tj} such that most entries are zero and nonzero entries are drawn from $\mathcal{N}(0, \sigma_d^2)$; then apply the centering normalization (53).
4. Form total utilities according to (51) and sample N_t i.i.d. choices per market from (52).

We then estimate (i) **DeepHalo-only** (no (μ, d)), (ii) **Halo+μ** (intercepts only, d disabled), and (iii) **Full Zhang-Sparse** (two-stage training with (μ, d) and ℓ_1).

Evaluation metrics and scope. We evaluate predictive fit via negative log-likelihood (NLL) computed on the *training data* (the same observations used to fit each model). There is no held-out test set: NLL here measures in-sample fit, not generalisation. An improvement in in-sample NLL is necessary but not sufficient evidence that the sparse shock layer recovers the true structure; the support recovery metrics below are the more diagnostic check. The stored smoke results (`results/hybrid/zhang_lu_sparse/test_smoke_sparse20`) use only 3 training epochs and show `sensitivity = 0.0`, meaning no nonzero entries are recovered by the thresholded $\hat{d}$. This is expected at 3 epochs; the full-run configuration (`epochs = 20`) is required for meaningful support recovery. For sparsity recovery we threshold $\hat{d}_{tj}$ to form a recovered support indicator

$$\hat{\gamma}_{tj} = \mathbb{1}\{|\hat{d}_{tj}| > \tau\}, \tag{55}$$

and compare it to the true support $\gamma_{tj}^\star = \mathbb{1}\{d_{tj}^\star \neq 0\}$. We report sensitivity and specificity:

$$\text{sensitivity} = \Pr(\hat{\gamma}_{tj} = 1 \mid \gamma_{tj}^\star = 1), \qquad \text{specificity} = \Pr(\hat{\gamma}_{tj} = 0 \mid \gamma_{tj}^\star = 0).$$

Correctness is indicated by (a) improved (or at least competitive) NLL for Zhang-Sparse relative to models that omit sparse shocks when sparse shocks are present in the data-generating process, and (b) a sensible threshold tradeoff: increasing τ decreases the predicted nonzero rate and typically increases specificity while reducing sensitivity.

Design choices and tradeoffs. The Zhang-Sparse extension requires explicit modeling choices because Lu(25) and Zhang(25) address different misspecifications. We choose an additive shock layer (51) rather than embedding shocks inside the DeepHalo representation because it preserves interpretability and allows direct validation of shock recovery in simulation. We use the decomposition $\xi_{tj} = \mu_t + d_{tj}$ rather than a dense free shock matrix because a dense ξ_{tj} would overfit and would contradict Lu(25)'s sparsity premise. We impose centering (53) rather than selecting an arbitrary reference product because centering is symmetric across products and stabilizes optimization. We use MAP with an ℓ_1 penalty rather than spike-and-slab MCMC in this subsection because the goal here is to validate the hybrid modeling idea and its integration with a deep context network; we reserve `tfp.mcmc` for Part 2(b), where MCMC is explicitly required. Finally, we adopt a two-stage training procedure to reduce confounding between context parameters and sparse shocks and to make support recovery easier to diagnose.

Implementation and reproducibility. The Zhang-Sparse model is implemented in `jpm_q3.hybrid.zhang_lu_sparse` using TensorFlow and `ChoiceDataset` (choice-learn) for data handling. Running the module produces two saved experiment folders (20% and 40% sparsity regimes), each containing `results.json`, `summary.csv`, and `support.csv`. These artifacts include the ablation NLL table and the τ-sweep support metrics used to validate the criteria above. We also include unit tests and smoke tests (Python `unittest`) that verify log-probability normalization, the centering identification constraint, simulation output consistency, and that smoke runs save artifacts and exhibit monotone threshold behavior (predicted nonzeros weakly decrease as τ increases).

16 Applicability of Lu(25) Sparsity to Credit Card Offer Demand

Lu & Shimizu (2025) replace the classical "price endogeneity + IV" identification logic with a different identifying restriction: market–product demand shocks have a sparse structure.[2] Question 3(e) asks whether that sparsity assumption is plausible in the credit card merchant-offer setting. The short answer is that sparsity is not impossible, but it is a strong assumption whose plausibility depends on (i) how the analyst defines markets, (ii) how complete the analyst's exposure and targeting covariates are, and (iii) whether residual unobservables are dominated by rare, discrete events or by diffuse personalization and marketing intensity. In most realistic data environments, the remaining unobservables that drive offer take-up are more likely to be *partially structured and correlated* than literally sparse, so a sparsity model should be treated as a targeted modeling choice that requires diagnostics rather than a default.

16.1 What sparsity means in Lu(25) and what it must explain

In the baseline demand equation,

$$\delta_{jt} \;=\; x_{jt}'\beta - \alpha p_{jt} + \xi_{jt}, \tag{56}$$

the unobserved shock ξ_{jt} represents any offer–market determinant of demand that is not captured in x_{jt} and is not absorbed by the heterogeneity structure used for share inversion. Lu(25) motivates a decomposition of the form

$$\xi_{jt} \;=\; \mu_t + d_{jt}, \qquad d_{jt} \text{ sparse over } (j,t), \tag{57}$$

where μ_t is a market-level intercept and d_{jt} is an idiosyncratic market–product deviation that is *exactly or approximately zero for most pairs*. This restriction is not merely a regularizer; it is the key source of identification when one does not rely solely on external instruments. If d_{jt} is not close to sparse in the relevant data scale, then the estimator can misattribute diffuse unobserved variation to β (and α), or can over-shrink true shocks, depending on prior/penalty strength.

Accordingly, the question "does sparsity apply" should be interpreted as: after conditioning on all observables that would reasonably be available to the analyst in a credit-offer system, is the remaining ξ_{jt} well approximated by a market intercept plus *rare* offer-specific spikes?

16.2 Why credit card offers often generate dense unobservables

Several mechanisms in credit card offer platforms naturally induce unobserved variation that is broad-based rather than sparse.

First, exposure and targeting can be pervasive. In many deployments, offer placement is continuously optimized (rankers, bandits, budget pacing, channel selection), and the econometrician may not observe the full exposure process (impressions, rank position, frequency caps, eligibility, suppression rules). If exposure is only partially measured, then ξ_{jt} absorbs missing exposure intensity. Because the platform is scoring and serving *many offers at once*, the unobserved exposure component can shift take-up probabilities across a large fraction of the offer set, producing a *dense* pattern in d_{jt} rather than a sparse one.

Second, personalization can create smooth residual heterogeneity. Even with rich consumer covariates, recommender systems typically use high-dimensional signals (purchase histories, embeddings, contextual bandits) that are not fully observable in standard choice-model datasets. The remaining mismatch between the analyst's features and the platform's latent scoring can act like a continuous, offer-specific residual across most offers. That behavior is closer to a low-rank or factor structure (latent tastes) than to sparsity.

Third, shocks may be correlated across subsets of offers. Merchant offers are naturally grouped (merchant category, geographic availability, partner networks, seasonality). Unobserved shifts such as holiday spend, macro conditions, or channel-level changes can move demand for *categories* of offers simultaneously. A market intercept μ_t absorbs a common shift, but correlated category-level residuals can remain, which again is not sparse in j.

These considerations do not imply that sparsity is impossible, but they imply that, in many realistic credit-offer datasets, ξ_{jt} is likely to contain a mixture of (i) market-level shifts, (ii) correlated group-level shifts, and (iii) diffuse residual personalization/exposure effects. A pure sparsity assumption for d_{jt} may therefore be too restrictive unless markets are defined in a way that isolates genuinely idiosyncratic events.

16.3 When a sparsity approximation could be plausible

There are scenarios where a sparsity approximation for d_{jt} is more defensible.

One plausible scenario is when markets are defined finely enough (e.g., *merchant-week* or *offer-week* within a narrow segment) and the analyst has strong controls for exposure and eligibility so that ξ_{jt} is dominated by rare

idiosyncratic merchant shocks. Examples include sudden merchant outages, localized negative press, a one-off viral trend, or a discrete change in redemption experience that affects only one merchant offer. In such cases, most offers in a given market may be "business as usual" while a small subset experiences genuinely large deviations, which is closer to sparsity.

A second scenario is when the platform is engineered such that only a small fraction of offers receive meaningful incremental exposure in a given period (for example, strict rotation constraints where only a few offers are actively promoted and others are effectively dormant). If the dataset is built at a level where these promotion decisions are not directly observed, then the unobserved promotion shocks could appear as sparse d_{jt}.

In both scenarios, the key requirement is that the analyst can credibly argue that, after conditioning on observed covariates, the remaining shocks are event-driven and localized rather than continuous and system-wide.

16.4 Practical diagnostics that would justify (or refute) sparsity

Because sparsity is an identifying restriction, applying Lu(25) in credit-offer data should be accompanied by diagnostics that speak directly to whether the fitted model is behaving "as if" shocks are sparse.

A first diagnostic is the empirical distribution of estimated deviations (or posterior inclusion probabilities in a Bayesian implementation). Under true sparsity, most d_{jt} should be near zero, and the model should place high mass on the spike component for most observations. If, instead, many d_{jt} are moderately nonzero and posterior inclusion probabilities cluster in the interior, that indicates diffuse residual variation inconsistent with sparsity.

A second diagnostic is stability across prior/penalty settings. If moderate changes in the sparsity prior (e.g., the Beta prior on inclusion probability or the spike variance) lead to large swings in $\hat{\alpha}$ (price sensitivity) or in recovered shocks, then the sparsity restriction is doing heavy identification work in a fragile way, which should reduce confidence in applying it as the primary identification strategy.

A third diagnostic is comparison to alternative structural restrictions, such as a low-rank factor model for ξ_{jt} or a control-function approach when some instruments or proxies are available. Agreement across fundamentally different identification strategies increases credibility; disagreement suggests that the structure imposed on ξ_{jt} is not well supported by the data.

16.5 Conclusion

In the credit card merchant-offer setting, it is plausible that some unobserved demand shocks are episodic and localized, which supports the *possibility* of a sparse approximation for d_{jt}. At the same time, common operational realities—partial observability of exposure, pervasive personalization, and correlated category-level variation—make it likely that the residual unobserved component is at least partly *dense* rather than sparse. For that reason, we view Lu(25)'s sparsity assumption as a potentially useful tool when markets are defined granularly and exposure controls are rich, but not as a default assumption for credit card offer demand estimation. In practice, applying the method should be paired with explicit sparsity diagnostics and robustness checks against alternative endogeneity corrections.

Part III

Bonus Question 1: Storable Goods, Stockpiling and Strategic Pricing

This part addresses the revised Bonus Question 1, which extends the static context-dependent choice model of Part 1 to a dynamic setting with stockpiling behaviour, endogenous prices, and consumer-specific unobserved tastes. The question is decomposed into five graded sub-parts; the structure below mirrors that decomposition.

17 Consumer Value Function

This section formalises the consumer's dynamic discrete choice problem. It defines the state variables, action space, transition dynamics, the Bellman equation incorporating Halo, endogenous prices, stockpiling, and consumer-specific unobserved tastes, and finally argues identifiability of each parameter.

17.1 Setup and notation

We consider a market with $J = 5$ brands selling a storable good over a finite horizon T. A panel of I consumers is observed. At each period $t \in \{1, \dots, T\}$, the platform presents a *common* choice set $\mathcal{A}_t \subseteq \{1, \dots, J\}$ with $|\mathcal{A}_t| \geq 3$; the set is identical for all consumers at time t but varies across periods. The outside option $j = 0$ (do not purchase) is always available. The total number of choice options is therefore $J + 1$.

The following structural assumptions are maintained throughout:

- Brands have no observable features beyond price. Differences between brands are captured by (i) a brand fixed effect α_j and (ii) an unobserved consumer-specific taste η_{ij} that is heterogeneous across consumers but *homogeneous across time*.
- Prices p_{jt} are endogenous: they are generated as a function of the unobserved market–product demand shock ξ_{jt} and an excluded observable cost shifter w_{jt} used only for the control-function correction in estimation.
- Each consumer i holds an integer inventory $s_{it} \in \{0, 1, \dots, S_{\max}\}$ that is observed.
- Consumption is fixed at one unit per period (Ching 2020).
- Each consumer has a personal discount factor $\delta_i \sim \text{Uniform}(\delta_{\min}, \delta_{\max})$ with $0 \leq \delta_{\min} < \delta_{\max} < 1$.
- Choice-specific errors ε_{ijt} follow i.i.d. Type-I Extreme Value (Gumbel) distributions.
- Consumers observe the full price path $\{p_{jt}\}_{t=1}^{T}$ from $t = 0$ onward and optimise their purchasing decisions accordingly. In the DGP, consumers also observe the demand shock ξ_{jt} directly (e.g., they experience product quality before purchase), so ξ_{jt} enters consumer utility in the backward induction. This is consistent with endogeneity being a problem for the *econometrician* — who observes prices but not ξ_{jt} — rather than for the consumer. Note that if consumers only observed prices and not ξ_{jt}, the price equation $p_{jt} = c_j + \gamma\xi_{jt} + \pi_w w_{jt} + \nu_{jt}$ would allow partial inference of ξ_{jt} from p_{jt} conditional on w_{jt}, but the noise term ν_{jt} means inference is imperfect; the direct observation assumption is adopted here for tractability.

17.2 Utility specification

The systematic instantaneous utility of consumer i choosing brand $j \geq 1$ at period t with inventory s is

$$\bar{u}_{ijt}(s) \;=\; \underbrace{\alpha_j}_{\text{brand FE}} \;+\; \underbrace{\beta\, p_{jt}}_{\text{price}} \;+\; \underbrace{h_j(\mathcal{A}_t)}_{\text{Halo}} \;+\; \underbrace{\eta_{ij}}_{\text{consumer taste}} \;+\; \underbrace{\xi_{jt}}_{\text{shock}}, \tag{58}$$

and for the outside option,

$$\bar{u}_{i0t}(s) \;=\; -\kappa_0\, \mathbf{1}\{s = 0\}, \tag{59}$$

where $\kappa_0 > 0$ is a stockout penalty making not purchasing costly when inventory is depleted. Total utility includes the Gumbel shock $u_{ijt}(s) = \bar{u}_{ijt}(s) + \varepsilon_{ijt}$.

Halo effect. The term $h_j(\mathcal{A}_t)$ is the context-dependent component: the attractiveness of brand j depends on the composition of the offered set, not only on j's own attributes. We parameterise it with the DeepHalo architecture of Part 1:

$$\big(h_j(\mathcal{A}_t)\big)_{j\in\mathcal{A}_t} \;=\; f_\theta(\mathcal{A}_t), \tag{60}$$

where f_θ is a permutation-equivariant neural network with parameters θ taking the availability vector as input.

Consumer-specific tastes η_{ij}. Per the question's structural assumption (2), η_{ij} is heterogeneous across consumers and constant over time. In the DGP we draw $\eta_{ij} \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma_\eta^2)$ for each (i, j) pair once at $t = 0$ and hold it fixed.

Market–product shocks ξ_{jt} (Lu-style). The unobserved demand shock decomposes as

$$\xi_{jt} \;=\; \mu_t + d_{jt}, \tag{61}$$

where $\mu_t \sim \mathcal{N}(0, \sigma_\mu^2)$ is a market-level mean shock and d_{jt} is a sparse brand-time deviation: $d_{jt} \overset{\text{i.i.d.}}{\sim} (1-\pi)\,\delta_0 + \pi\,\mathcal{N}(0, \sigma_d^2)$ (spike-and-slab).

Endogenous prices. Prices are generated as

$$p_{jt} \;=\; c_j + \gamma\,\xi_{jt} + \pi_w w_{jt} + \nu_{jt}, \qquad \nu_{jt} \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma_\nu^2), \tag{62}$$

where $c_j > 0$ is a brand-specific cost shifter and $\gamma > 0$ controls the endogeneity strength. The term w_{jt} is an observable exogenous brand-time cost shifter excluded from utility; in the implementation $\pi_w = 1$. It is introduced solely to identify the price coefficient through the Petrin & Train (2010) control function in Section 18.4. Because p_{jt} and ξ_{jt} share the common factor $\gamma\xi_{jt}$, endogeneity is present by construction.

17.3 State, action, transition

State. The payoff-relevant state for consumer i at period t is (s_{it}, t) where $s_{it} \in \{0, 1, \ldots, S_{\max}\}$ is integer inventory and $t \in \{1, \ldots, T\}$ indexes calendar time. Because consumers see the full price path, future prices do not constitute an additional state variable. The consumer's identity-specific tastes $\eta_{i\cdot}$ and discount δ_i are time-invariant parameters of consumer i's problem, not states.

Action. At each period the consumer chooses $a_{it} \in \mathcal{A}_t \cup \{0\}$: either purchase one unit of an available brand ($a_{it} = j \geq 1$) or abstain ($a_{it} = 0$).

Inventory transition. One unit is consumed per period regardless of purchase (Ching 2020). Letting $s_{\text{cons}} = \max(0, s_{it} - 1)$:

$$s_{i,t+1} \;=\; \begin{cases} \min(S_{\max},\; s_{\text{cons}} + 1) & \text{if } a_{it} \geq 1, \\ s_{\text{cons}} & \text{if } a_{it} = 0. \end{cases} \tag{63}$$

17.4 Bellman equation

Because ε_{ijt} are i.i.d. Gumbel, the expectation over ε in the dynamic programming problem admits a closed-form log-sum-exp expression (Rust 1987). Let $V_t^i(s)$ denote the value function for consumer i at period t with inventory s. The **Bellman equation** is

$$\boxed{V_t^i(s) \;=\; \log \sum_{j\in\mathcal{A}_t\cup\{0\}} \exp\Big(\bar{u}_{ijt}(s) \;+\; \delta_i\, V_{t+1}^i(s'(j,s))\Big)} \tag{64}$$

where $s'(j, s)$ is the deterministic next inventory from (63).

Terminal condition.

$$V_{T+1}^i(s) \;=\; 0 \qquad \forall\, s \in \{0, \ldots, S_{\max}\}. \tag{65}$$

Optimal choice probability. Integrating the Gumbel shock, the probability that consumer i chooses brand j at (s, t) is

$$P(a_{it} = j \mid s_{it} = s,\, \mathcal{A}_t,\, p_t;\, \theta) \;=\; \frac{\exp\big(\bar{u}_{ijt}(s) + \delta_i V^i_{t+1}(s'(j,s))\big)}{\sum_{k \in \mathcal{A}_t \cup \{0\}} \exp\big(\bar{u}_{ikt}(s) + \delta_i V^i_{t+1}(s'(k,s))\big)}. \tag{66}$$

In the DGP we solve (64) exactly by backward induction in pure numpy for each consumer, then forward-sample choices according to (66). In estimation $V^i_{t+1}(s'(j,s))$ is approximated by a learned neural value head; the approximation is detailed in Section 18.

17.5 Parameter identifiability

The unknown DGP parameters are $\theta = (\{\alpha_j\}_{j=1}^J,\, \beta,\, \{\eta_{ij}\}_{i,j},\, \{\mu_t\}_{t=1}^T,\, \{d_{jt}\},\, \pi,\, \{\delta_i\}_{i=1}^I,\, \kappa_0,\, f_\theta)$. We discuss identification, normalisation, or estimation treatment of each below.

Brand fixed effects α_j. Identified from cross-brand variation in average choice frequencies conditional on the choice set, after controlling for prices and shocks. Repeated appearance of brand j across many different sets $\mathcal{A}_t$ identifies its location parameter α_j. The normalisation $\alpha_0 = 0$ (outside option) is imposed implicitly via the masked log-softmax.

Price coefficient β. Identified from within-brand variation in p_{jt} across time, controlling for α_j, h_j, η_{ij} and ξ_{jt}. However, because p_{jt} is endogenous via (62), a MAP estimator that does not correct for endogeneity will be biased toward zero. Our simulation study (Section 19) demonstrates this bias deliberately. Consistent estimation requires either instrumental variables (cost shifters c_j, in the BLP tradition) or the sparse-shock approach of Lu & Shimizu (2025) used in Part 2.

Consumer-specific tastes η_{ij}. Identified from within-consumer variation across periods. Consumer i is observed making T choices; the systematic tendency to favour brand j across multiple choice sets, after removing brand FE, price, halo and ξ_{jt}, identifies η_{ij}. Identification is panel-style: each consumer contributes T observations and J taste parameters, so a necessary condition is $T \gtrsim J$, which is satisfied in our DGP ($T = 20$, $J = 5$).

Market mean shocks μ_t. Identified from market-level demand variation: after controlling for brand FE, prices, and consumer tastes, the average residual utility across inside goods at time t identifies μ_t.

Sparse deviations d_{jt}. Identified from brand-time residual variation after removing μ_t and η_{ij}. The sparsity assumption (π small) constrains most d_{jt} to zero; non-zero entries are identified by brands whose demand deviates systematically from both the market mean and consumer-specific tastes at specific periods.

Sparsity parameter π. Identified from the empirical fraction of non-zero d_{jt} entries. With large TJ, the proportion of brand-time pairs with significant residual demand converges to π.

Discount factor δ_i. Intertemporal substitution is informative about patience: a consumer with high δ_i stockpiles more aggressively when prices are low. Variation in inventory holdings across consumers facing the same price paths therefore contains information about the distribution of δ_i. However, dynamic discrete-choice discount factors are not generally point-identified without additional normalisations or exclusion restrictions. In the estimator we therefore do not estimate each δ_i; we use the stated modelling simplification of a common fixed $\delta = 0.825$, the mean of the generating distribution.

Stockout penalty κ_0. Identified from emergency-purchase behaviour: when $s_{it} = 0$, a larger κ_0 raises the probability of purchasing at any price. The difference in purchase rates between $s_{it} = 0$ and $s_{it} > 0$ identifies κ_0.

Halo parameters f_θ. Identified from menu variation: systematic changes in $P(j \mid \mathcal{A}_t)$ across different sets that cannot be explained by price, α_j, η_{ij} or ξ_{jt} are attributed to the Halo network. Permutation equivariance of the DeepHalo architecture ensures the learned function depends on the set composition, not ordering.

Summary. The static utility, shock, Halo, stockout, and price parameters are identified under the maintained panel, menu-variation, sparsity, and cost-shifter assumptions above. The heterogeneous discount factors require additional normalisation, so the implemented estimator fixes a common discount rather than trying to recover δ_i separately. Finite-sample recovery quality depends on the panel dimensions (I, T, J) and on whether endogeneity is corrected; Section 19 reports recovery results from a single simulation draw.

18 Estimation Algorithm

This section derives the estimation algorithm for recovering the model parameters defined in (58) and the Bellman equation (64). We implement the estimator in `jpm_q3.bonus1.dynamic_model` using TensorFlow and TensorFlow Probability. All computationally intensive operations execute in graph mode without retracing, satisfying the JPM constraint.

18.1 Modelling choices and their justifications

The revised question explicitly grants flexibility: *"You are free to make any other assumptions if there are ambiguities."* We document each modelling choice below so the trade-offs are on the record.

1. **Common discount δ in estimation** (vs. heterogeneous δ_i in the DGP). Magnac & Thesmar (2002) show that discount factors are not point-identified without additional structure in single-agent dynamic discrete choice. The standard approach in applied work (e.g. Rust 1987) is to fix δ at a reasonable value. We use $\delta = 0.825$, the mean of the true generating distribution Uniform$(0.70, 0.95)$. This choice minimises the misspecification bias relative to the DGP while remaining consistent with the non-identification result.

2. **Shared value function $V_\psi(t, s)$ across consumers.** A consumer-specific $V_\psi(t, s, i)$ would inflate parameter count without substantial benefit since η_{ij} is already in static utility (Hendel & Nevo 2006 make the same approximation).

3. **Observable cost shifter w_{jt} for the Petrin & Train (2010) control function.** The question states that brands have no observable features beyond prices. To enable identification of β_{price} under endogeneity, we extend the DGP with an observable brand-time cost shifter w_{jt} representing exogenous input cost shocks (commodity prices, exchange rates, supplier markups). This is consistent with the question's explicit allowance for additional assumptions and is the standard practice in industrial organisation when estimating demand elasticities (Berry-Levinsohn-Pakes 1995). Without an exogenous instrument, β_{price} is not point-identified.

4. **Spike-and-slab on d implemented as a MAP penalty rather than full MCMC.** Lu & Shimizu (2025) use Metropolis sampling over spike-and-slab indicators; we adopt a MAP approximation via `tfd.MixtureSameFamily` for tractability inside a deep model trained by stochastic gradient descent.

5. **Two-stage training with static NLL for Stage 1.** Prevents the randomly-initialised value-network embedding Embed(t) from absorbing the time-period signal that should be attributed to μ_t. The Stage 1 objective is mathematically equivalent to setting $\delta = 0$ during the first phase only.

6. **Negative initialisation $\beta_{\text{price}} = -1.0$.** The loss landscape has a flat region around $\beta = 0$ where the control-function coefficient λ_{control} and the free parameters (μ, d, η) collectively absorb the price signal, leaving β trapped near zero. Initialising at a moderately negative value (consistent with the economic prior that higher price reduces utility) escapes this basin without biasing the converged estimate toward the truth.

7. **TF 2.16/Keras 3 embedding-gradient limitation.**[3] This is a known limitation of the TF/Keras version used and is not a flaw in the estimation algorithm itself.

18.2 MAP objective

We estimate by Maximum A Posteriori (MAP). The parameter vector estimated in the deep model is

$$\theta_{\text{est}} = \big(\beta,\ \lambda_{\text{control}},\ \mu,\ d,\ \text{logit}(\pi),\ \eta,\ \psi_{\text{Halo}},\ \psi_V\big), \tag{67}$$

where ψ_{Halo} are the DeepHalo weights and ψ_V are the value-network weights. There is no separate trainable α_j variable in the estimator: the featureless DeepHalo item embedding supplies brand-specific baseline utility and context effects, with the outside option normalised through the masked log-softmax. The consumer-specific discount δ_i is approximated by a common δ fixed at 0.825 during estimation (the true δ_i are heterogeneous in the DGP; this is a deliberate modelling simplification documented in Section 17).

[3] In TensorFlow 2.16 with Keras 3, integer-indexed `keras.Variable` embedding lookups inside a `@tf.function`-compiled training step do not receive gradient updates through `GradientTape` auto-watching. Consequently, the DeepHalo item embedding (ψ_{Halo}) and the value-head market embedding (Embed(t)) receive zero gradient during Stage 2's compiled step, despite being nominally unfrozen. In practice this has minimal impact on the econometric parameters of interest (β, λ, μ, d, η): item IDs are constant across all observations ($\{0, 1, \ldots, J\}$ always present), so the Halo item embedding acts as a fixed per-item bias absorbed into μ and η during Stage 1. The value-head MLP weights (non-embedding layers) receive correct gradients in Stage 2 and are confirmed to change by `test_stage2_changes_value_head_weights`.

The MAP objective is

$$\mathcal{L}(\theta_{\text{est}}) \;=\; \mathcal{L}_{\text{NLL}}(\theta_{\text{est}}) \;+\; w_{\text{TD}}\,\mathcal{L}_{\text{TD}}(\theta_{\text{est}}) \;+\; w_{\text{prior}}\,\mathcal{L}_{\text{prior}}(\theta_{\text{est}}), \tag{68}$$

with the negative log-likelihood

$$\mathcal{L}_{\text{NLL}}(\theta_{\text{est}}) \;=\; -\frac{1}{N}\sum_{n=1}^{N} \log P(y_n \mid s_n, \mathcal{A}_{t(n)}, p_{t(n)};\, \theta_{\text{est}}), \tag{69}$$

where $P(\cdot)$ is given by (66) with V^i_{t+1} replaced by the value network $V_{\psi_V}(t+1, s')$.

The Bellman consistency term is implemented as a temporal-difference penalty:

$$\mathcal{L}_{\text{TD}}(\theta_{\text{est}}) \;=\; \frac{1}{N}\sum_{n=1}^{N} \Big[V_{\psi_V}(t_n, s_n) - \big(r_n + \delta(1 - d_n^{\text{done}}) V_{\psi_V}(t_{n+1}, s_{n+1})\big)\Big]^2. \tag{70}$$

In code, the default weight is $w_{\text{TD}} = 0.1$ (`DynamicModelConfig.td_weight`). Stage 1 sets this contribution to zero by using the static objective below; Stage 2 includes it in the graph-mode MAP step.

18.3 Negative log prior via `tfp.distributions`

The negative log prior is a sum of four terms, each implemented with a TensorFlow Probability distribution rather than hand-rolled formulas:

$$\mathcal{L}_{\text{prior}} \;=\; \mathcal{L}_d^{\text{spike-slab}} \;+\; \mathcal{L}_\pi^{\text{Beta}} \;+\; \mathcal{L}_\mu^{\text{Gaussian}} \;+\; \mathcal{L}_\eta^{\text{Gaussian}}. \tag{71}$$

Spike-and-slab on d_{jt}. We use `tfd.MixtureSameFamily` to compose two `tfd.Normal` components weighted by a `tfd.Categorical`:

$$p(d_n \mid \pi) = (1-\pi)\,\mathcal{N}(d_n; 0, \sqrt{v_0}) + \pi\,\mathcal{N}(d_n; 0, \sqrt{v_1}), \tag{72}$$

with spike variance v_0 and slab variance $v_1 \gg v_0$.

Beta prior on π. Since $\pi \in (0,1)$ is parameterised in logit space, we use `tfd.Beta(concentration1 =` a_π`, concentration0 =` b_π`)` and include the logistic Jacobian for the change of variables:

$$\mathcal{L}_\pi^{\text{Beta}} \;=\; -\big(\log p_{\text{Beta}}(\pi) + \log\pi + \log(1-\pi)\big). \tag{73}$$

Gaussian priors on μ_t and η_{ij}. Both are implemented with `tfd.Normal`:

$$\mathcal{L}_\mu^{\text{Gaussian}} \;=\; -\frac{1}{T}\sum_{t=1}^{T} \log\mathcal{N}(\mu_t; 0, \sigma_\mu), \qquad \mathcal{L}_\eta^{\text{Gaussian}} \;=\; -\frac{1}{IJ}\sum_{i,j} \log\mathcal{N}(\eta_{ij}; 0, \sigma_\eta). \tag{74}$$

The Gaussian prior on η_{ij} regularises the $I \times J$ fixed effects.

18.4 Petrin & Train (2010) control function for price endogeneity

Without an explicit correction, the MAP estimator's $\hat{\beta}_{\text{price}}$ is biased toward zero because p_{jt} is correlated with the unobserved shock ξ_{jt} by construction (Section 17). We apply the canonical correction for discrete-choice endogeneity from Petrin and Train [4].

Why Petrin & Train and not BLP IV. Berry-Levinsohn-Pakes (1995) IV requires share inversion to linearise the demand system, which is incompatible with our individual-level panel containing η_{ij} and the DeepHalo non-parametric backbone. The control-function approach operates directly on the original utility specification, adding one scalar parameter and one residual term. Petrin & Train remains the standard reference for this setting: implementable, interpretable, and theoretically grounded.

First stage. We project the endogenous price onto observable exogenous instruments. With the cost shifter w_{jt} as the excluded instrument:

$$p_{jt} \;=\; \tau_j \;+\; \pi_1 w_{jt} \;+\; \pi_2 w_{jt}^2 \;+\; \hat{e}_{jt}, \tag{75}$$

where τ_j are brand fixed effects in the first stage (separate from α_j in utility) and $\hat{e}_{jt}$ is the OLS residual. The first stage is computed once in pure NumPy by closed-form OLS: `control_function.compute_price_residuals`.

Second stage. We include $\hat{e}_{jt}$ in utility as a control:

$$u_{ijt}^{(\mathrm{CF})} \;=\; \alpha_j + \beta_{\text{price}}\, p_{jt} \;+\; \boxed{\lambda_{\text{control}}\, \hat{e}_{jt}} \;+\; h_j(\mathcal{A}_t) + \eta_{ij} + \xi_{jt}. \tag{76}$$

Under the control-function assumption $\xi_{jt} = \rho\hat{e}_{jt} + \omega_{jt}$ with $\omega_{jt} \perp \hat{e}_{jt}$, the estimated $\hat{\lambda}_{\text{control}}$ absorbs the endogenous component and $\hat{\beta}_{\text{price}}$ becomes consistent for the structural price effect. Both coefficients are trainable parameters of the deep model (`model.beta_price`, `model.lambda_control`).

Identification of λ_{control}. At small sample sizes ($T \lesssim 10$, $I \lesssim 50$), $\hat{\lambda}_{\text{control}}$ can be near zero even when the DGP has $\gamma > 0$, because the model has many flexible parameters (μ, d, η, Halo weights) that collectively absorb price-correlated variation before λ_{control} has a chance to do so. At the published DGP scale ($T = 20$, $I = 150$, $E_1 = 30$ epochs), the single-draw estimate is $\hat{\lambda}_{\text{control}} = +0.286$ (theoretically expected positive sign under $\gamma > 0$), but the multi-seed coverage study at the smaller scale shows near-zero values. Users should treat the control function as providing partial correction rather than full identification, particularly in short panels.

Standard errors. The Laplace credible interval for $\hat{\beta}_{\text{price}}$ from the exact MAP Hessian remains valid in the second stage *conditional on* the first-stage residuals being correct. A fully rigorous analysis would adjust standard errors via the Hahn-Schmid two-stage formula; for the present submission we report the Laplace intervals from the full MAP objective and note this caveat in Section 19.

18.5 Identification constraint and centering

To avoid the additive non-identification of (μ_t, d_{jt}) ($\mu_t + c$ and $d_{jt} - c$ give the same total shock), we impose within-market centering of d_{jt} at the forward-pass level:

$$d_{tj} \;\leftarrow\; d_{tj} - \tfrac{1}{J}\sum_{k=1}^{J} d_{tk}. \tag{77}$$

This is done in `model._augmented_utilities` and ensures μ_t retains the interpretation of a market-level mean shock.

18.6 Two-stage training

A direct joint optimisation of θ_{est} allows the value network V_{ψ_V} (which uses an embedding of t) to absorb time-period variation that should be attributed to μ_t, biasing $\hat{\mu}_t$ toward zero. To isolate the econometric parameters from the value-network approximation, we employ a two-stage procedure.

Stage 1 (econometric estimation). Freeze the Halo and value-network weights. Train only the econometric parameters ($\beta, \lambda_{\text{control}}, \mu, d, \eta, \operatorname{logit}\pi$) using a *static* NLL that omits the continuation value entirely:

$$\mathcal{L}_{\text{NLL}}^{\text{static}} \;=\; -\frac{1}{N}\sum_{n} \log \operatorname{softmax}_{j\in\mathcal{A}_{t(n)}\cup\{0\}}\big(\bar{u}_{ij\,t(n)}(s_n)\big). \tag{78}$$

This removes the random initialisation of V_{ψ_V} as a source of bias. In code, this corresponds to `model.static_choice_nll(...)` in `model.py`.

Stage 2 (joint fine-tuning). Unfreeze all parameters and train jointly using the full dynamic NLL, the Bellman/TD penalty, and the prior. The Stage 2 step uses the same MAP objective (68), with $\mathcal{L}_{\text{NLL}}$ now including the continuation value and $\mathcal{L}_{\text{TD}}$ enforcing value-function consistency.

Implementation note: embedding gradient limitation in TF 2.16/Keras 3. In TensorFlow 2.16 with Keras 3, `keras.Variable` embedding lookups (indexed by integer tensors) do *not* receive gradients through `@tf.function`'s `GradientTape` auto-watching, even when the layer is marked trainable. In practice, Stage 2 therefore updates: (i) the value head ψ_V (dense MLP, confirmed by `test_stage2_changes_value_head_weights`); (ii) the scalar/vector econometric parameters $\beta, \mu, d, \eta, \lambda$. The Halo item embedding and the market embedding ψ_{Halo} do not receive gradient updates and remain at their Stage 1 (frozen/random-init) values. Because the item IDs are constant across all observations in the simulation ($j \in \{0, 1, 2, 3\}$ for all t and i), the Halo output reduces to a fixed per-item bias that is absorbed into μ_t and η_{ij} during Stage 1; Stage 2's value of learning ψ_{Halo} is therefore limited in this

simulation design regardless of the gradient issue. The primary results (recovery of β_{price}, μ_t, η_{ij}) are unaffected, as these parameters are scalar/vector variables that receive gradients correctly in both stages.

Algorithm 1 Two-stage MAP estimation (revised Bonus 1)

Require: Data $\{(s_n, \mathcal{A}_{t(n)}, p_{t(n)}, y_n)\}_{n=1}^{N}$, priors $(v_0, v_1, a_\pi, b_\pi, \sigma_\mu, \sigma_\eta)$, epochs (E_1, E_2), learning rate η_{lr}

Initialise θ_{est} (zero for μ, d, η; random for $\psi_{\text{Halo}}, \psi_V$)
Stage 1: freeze $\psi_{\text{Halo}}, \psi_V$
for $e = 1, \ldots, E_1$ **do**
  for batch $\mathcal{B}$ in dataset **do**
    $\mathcal{L} \leftarrow \mathcal{L}_{\text{NLL}}^{\text{static}}(\mathcal{B}) + w_{\text{prior}}\, \mathcal{L}_{\text{prior}}$
    Update $(\beta, \lambda_{\text{control}}, \mu, d, \eta, \text{logit}\, \pi)$ via Adam
  end for
end for
Stage 2: unfreeze all (note: Halo/market embeddings receive zero gradients in TF 2.16/Keras 3 — see implementation note above)
for $e = 1, \ldots, E_2$ **do**
  for batch $\mathcal{B}$ in dataset **do**
    $\mathcal{L} \leftarrow \mathcal{L}_{\text{NLL}}(\mathcal{B}) + w_{\text{TD}}\, \mathcal{L}_{\text{TD}}(\mathcal{B})$
      $+ w_{\text{prior}}\, \mathcal{L}_{\text{prior}}$
    Update $\psi_V, \beta, \mu, d, \eta, \lambda, \text{logit}\, \pi$ via Adam (smaller lr)
  end for
end for
return $\hat{\theta}_{\text{est}}$

18.7 Graph-mode execution without retracing

The JPM constraint requires that all computationally intensive operations run in graph mode without retracing. We achieve this through three design choices.

1. Input-signature on `train_step`. `DynamicTrainer` compiles the training step via `tf.function(input_signature=[_sig])` where `_sig` is a dict of `tf.TensorSpec` entries with `None` batch dimensions:

```
_sig = {
  "item_ids":          tf.TensorSpec([None, J], tf.int32),
  "available":         tf.TensorSpec([None, J], tf.float32),
  "price":             tf.TensorSpec([None, J], tf.float32),
  "market_id":         tf.TensorSpec([None],    tf.int32),
  "household_id":      tf.TensorSpec([None],    tf.int32),
  "inventory":         tf.TensorSpec([None],    tf.float32),
  "choice":            tf.TensorSpec([None],    tf.int32),
  "reward":            tf.TensorSpec([None],    tf.float32),
  "done":              tf.TensorSpec([None],    tf.float32),
  "next_item_ids":     tf.TensorSpec([None, J], tf.int32),
  "next_available":    tf.TensorSpec([None, J], tf.float32),
  "next_price":        tf.TensorSpec([None, J], tf.float32),
  "next_market_id":    tf.TensorSpec([None],    tf.int32),
  "next_household_id": tf.TensorSpec([None],    tf.int32),
  "next_inventory":    tf.TensorSpec([None],    tf.float32),
}
```

The `None` batch dimension means a single traced graph handles any batch size, eliminating retraces between the regular batches and the (smaller) final batch of each epoch.

2. Frozen Python booleans. The within-market centering flag is captured once at construction time as a plain Python bool (`self._center_d`). Reassigning the configuration object during the two-stage procedure does not change the value seen inside `call()`, so the traced graph is stable across stages.

3. `-inf` **sentinel in masked softmax.** Unavailable items receive `tf.constant(float('-inf'))` as their logit before the softmax. This gives *exactly* zero probability rather than the approximate zero from a -10^9 sentinel. The advantage beyond numerical exactness is debugging: any bug that places a finite value on an unavailable item surfaces immediately as a `NaN`, rather than being masked by a near-zero epsilon.

18.8 Credible intervals via the exact MAP Hessian

Given the MAP estimate $\hat{\theta}_{\text{est}}$, we approximate the posterior by a Gaussian $p(\theta \mid \text{data}) \approx \mathcal{N}(\hat{\theta}, H^{-1})$ where $H = \nabla^2_\theta \mathcal{L}(\hat{\theta})$ is the Hessian of the full MAP objective (NLL + TD + prior). The prior contributes curvature $1/\sigma^2_\mu$ per μ element and $1/\sigma^2_\eta$ per η element, preventing degenerate intervals when the likelihood is flat.

For each parameter, we compute the exact *diagonal* of H via automatic differentiation. The standard error of θ_i is $\text{SE}(\theta_i) = 1/\sqrt{|H_{ii}|}$, and the 95% credible interval is $\hat{\theta}_i \pm 1.96 \cdot \text{SE}(\theta_i)$.

Scalar parameters. For scalar parameters (β, $\text{logit}\,\pi$), the diagonal is the second derivative, computed via nested `tf.GradientTape`:

```
with tf.GradientTape() as t2:
    with tf.GradientTape() as t1:
        loss = map_loss()
    g = t1.gradient(loss, param)
h_diag = t2.gradient(g, param)   # scalar second derivative
```

Vector and matrix parameters. For $\mu \in \mathbb{R}^T$ and $d \in \mathbb{R}^{T \times J}$, we use `t2.jacobian` to compute the full Hessian and extract its diagonal. The matrix case flattens d to a TJ-vector, computes the Jacobian of the gradient, and reshapes the diagonal back to $T \times J$.

Single compiled graph. All four Hessian computations are bundled inside a single zero-argument `@tf.function` wrapper that closes over the loss function and model parameters. This satisfies the no-retracing constraint: the interval computation traces exactly once.

18.9 Implementation map

The estimator components live in the following files:

Component	File
Model + losses	`src/jpm_q3/bonus1/dynamic_model/model.py`
Training loop (Stage 1 + Stage 2)	`simulation_study.py`
Generic graph-mode trainer	`trainer.py`
Laplace credible intervals	`intervals.py`
Petrin & Train (2010) control function	`control_function.py`
Counterfactual analysis (Part 4)	`counterfactual.py`
Configuration	`config.py`
Synthetic data generator	`data.py`

Smoke and unit tests in `tests/bonus/test_bonus1_dynamic_smoke.py` verify: DGP correctness (common $\mathcal{A}_t$, integer inventory, endogenous prices, η_{ij} shape and homogeneity), model correctness (output shapes, probability normalisation, float32 weights, η trainability), graph-mode correctness (`compile_train_step=True` runs without error), and counterfactual correctness (revenue and share computed for both baseline and promotion).

19 Simulation and Parameter Recovery

This section presents a simulation study that validates the estimation procedure on synthetic data with known ground-truth parameters. We report point estimates alongside 95% Laplace credible intervals for every estimated DGP parameter and discuss the structural identification limits at the chosen DGP scale before turning to the results.

19.1 Identification limits at $T = 20$, $I = 150$

Key caveat stated upfront. The deep model has many flexible parameters ($I \times J = 750$ consumer tastes η_{ij}, $T = 20$ market shocks μ_t, $T \times J = 100$ sparse deviations d_{jt}, plus Halo network weights). At $T = 20$, $I = 150$ (N = 3,000 observations), these parameters collectively absorb most price-driven and time-driven variation before the scalar parameters β_{price}, λ_{control}, π, κ_0 have a chance to acquire identifying curvature. As a result, the MAP Hessian diagonal for all four scalar parameters is near zero at the converged estimate, yielding standard errors of order 1–5 and 95% credible intervals that span a wide range including zero.

This is a structural feature of the estimation problem, not an implementation error:

- To narrow the $\hat{\beta}_{\text{price}}$ CI below zero would require SE $< |\hat{\beta}|/1.96 \approx 0.45$, implying $|H_{\beta\beta}| > 4.9$. The current $|H_{\beta\beta}| \approx 0.37$ suggests roughly $4.9/0.37 \approx 13\times$ more data is needed (all else equal), i.e. $I \gtrsim 2{,}000$.
- The same weak-identification pattern applies to κ_0 (stockout penalty), λ_{control} (CF coefficient), and π (sparsity fraction).
- The directional results — corr$(\hat{\mu}, \mu^\star)$ and RMSE$(\hat{\eta})$ — are *not* subject to this scalar identification problem and are the informative outputs of the study.

The simulation study therefore has two goals: (i) confirm the pipeline is mechanically correct end-to-end, and (ii) document which parameters are identified in direction/rank order at this scale and which require more data for magnitude recovery.

19.2 Note on brand fixed effects and Halo weights

The brand fixed effects α_j are not reported separately because they are encoded in the featureless DeepHalo item embedding and are not a distinct scalar parameter in the estimator. The Halo network weights ψ_{Halo} are also not compared element-wise to the DGP truth: the DGP Halo and the estimator Halo are two independently randomly-initialised networks; there is no canonical alignment between their weight tensors (analogous to the rotation indeterminacy in factor models). The informative Halo diagnostic is the menu-dependent choice probability, not weight matching. A quantitative out-of-sample prediction check is left for future work.

19.3 Experimental setup

The synthetic panel is generated by `jpm_q3.bonus1.dynamic_model.data.simulate_dynamic_panel` using the DGP described in Section 17. True parameter values for the reported run:

Parameter	True value
$\beta^\star_{\text{price}}$	-1.500
$\kappa^\star_0$ (stockout penalty)	2.000
$\sigma^\star_\mu$ (std of μ_t)	1.000
$\sigma^\star_d$ (std of non-zero d_{jt})	0.800
$\sigma^\star_\eta$ (std of η_{ij})	0.500
γ (endogeneity strength)	0.500
$\pi^\star$ (sparsity)	implied ≈ 0.20
$\delta^\star_i$ range	$[0.70, 0.95]$
J (brands)	5
T (periods)	20
I (consumers)	150
$S_{\max}$	5

Training uses the two-stage procedure of Algorithm 1 with $E_1 = 30$ epochs (Stage 1, static NLL) and $E_2 = 10$ epochs (Stage 2, joint fine-tuning at reduced learning rate). Initialisation: $\beta = -1.0$, $\kappa_0 = 1.0$ (intentionally offset from truth), zero for μ, d, η. True Halo weights from the DGP are saved in metadata but *not* given to the estimator. All scalar parameters including κ_0 are included in the Stage 1 econometric training set. Results stored in `results/bonus1/simulation_study/simulation_study.json` (with CF) and `simulation_study_no_cf.json` (without CF).

19.4 Recovery results

Table 3 reports the point estimates and 95% credible intervals against ground truth (single DGP draw, seed = 0). As described in Section **??**, scalar parameter CIs are wide by construction at this DGP scale; the Table presents them fully rather than suppressing them, so the reader can assess identification directly.

Table 3: Parameter recovery: true vs. estimated values with 95% Laplace credible intervals (single DGP draw, seed = 0). Standard errors are SE $= 1/\sqrt{|H_{ii}|}$ from the MAP Hessian diagonal (exact automatic differentiation). All scalar parameters have wide CIs reflecting weak identification at $N = 3{,}000$; see Section **??**. The "Without CF" column uses the model without the control function; "With CF" uses the Petrin & Train (2010) correction. Both columns use $\delta = 0.825$ (mean of true Uniform$(0.70, 0.95)$) and seed = 0.

Parameter	True	Without CF	With CF	95% CI (With CF)
β_{price}	-1.500	-0.779	-0.881	$[-4.11, +2.34]$
λ_{control}	—	—	$+0.324$	$[-6.72, +7.37]$
π (sparsity)	~ 0.20	0.094	0.096	[0.00, 1.00]
κ_0 (stockout penalty)	2.000	1.332	1.340	$[-8.02, +10.70]$
std$(\hat{\mu})$	0.935	0.146	0.219	—
corr$(\hat{\mu}, \mu^\star)$	1.000	0.814	0.737	—
RMSE$(\hat{\eta} - \eta^\star)$	—	0.447	0.476	$\approx \sigma_\eta^\star$

What the CI column tells us. Every scalar parameter has a 95% CI that spans zero (or effectively the entire feasible range for $\pi \in [0, 1]$). The MAP Hessian diagonals are $|H_{\beta\beta}| \approx 0.37$, $|H_{\kappa\kappa}| \approx 0.044$, $|H_{\lambda\lambda}| \approx 0.078$, $|H_{\pi_\ell \pi_\ell}| \approx 0.60$ (logit scale). All are small, confirming that the loss is nearly flat at the MAP point with respect to these parameters. The 95% CI for every scalar parameter contains the true value, so coverage is technically 100%; this is not statistical calibration but a consequence of interval width.

Control function: right direction, modest magnitude. The CF correction moves $\hat{\beta}_{\text{price}}$ from -0.779 (without CF, recovering 52% of true magnitude) to -0.881 (with CF, recovering 59%), a +7-percentage-point improvement in the recovered fraction. The CF coefficient $\hat{\lambda}_{\text{control}} = +0.324$ has the theoretically correct positive sign under $\gamma > 0$ (high residual price correlates with high unobserved demand, by construction). The modest improvement (rather than full recovery of the true -1.5) is expected: the deep model's flexible parameters absorb much of the price variation, leaving the CF term with a narrow identifying window.

μ_t recovery: direction is identified, scale is shrunk. The MAP estimator shrinks std$(\hat{\mu})$ toward zero (0.219 vs true 0.935) but preserves the ordinal relationship: corr$(\hat{\mu}, \mu^\star) = 0.737$ with the CF correction. This is the standard Stein-James MAP shrinkage under a Gaussian prior with weak likelihood signal: direction (rank order) is identified, magnitude is not. The positive correlation confirms that the model attributes the correct relative market-level shocks to the correct periods.

η_{ij} recovery near the noise floor. RMSE$(\hat{\eta} - \eta^\star) = 0.476 \approx \sigma_\eta^\star = 0.500$. The panel structure ($T = 20$ observations per consumer) provides enough information to recover consumer-brand tastes at the noise floor.

κ_0 (stockout penalty): first estimation, direction correct. The stockout penalty is now included in Stage 1 training and reported for the first time. Both with and without CF, $\hat{\kappa}_0 \approx 1.34$ (true = 2.0), recovering 67% of the true magnitude. The CI $[-8.02, +10.70]$ contains the truth but is very wide, consistent with the weak identification pattern described above: the stockout penalty is identified only from the difference in purchase rates at $s_{it} = 0$ vs $s_{it} > 0$, which is a sparse signal ($S_{\max} = 5$ means stockout at $s = 0$ is relatively rare).

19.5 Interpretation

End-to-end correctness is the primary result. The simulation establishes that the data-generation, two-stage estimation, Petrin & Train control function, Laplace intervals, and counterfactual analysis all run without numerical failure (graph mode, no retracing). This is the primary validation purpose of the study.

Identified vs. weakly-identified parameters. At $T = 20$, $I = 150$:

- **Identified in direction**: μ_t (corr = 0.737) and η_{ij} (RMSE at noise floor).
- **Weakly identified**: β_{price}, κ_0, λ_{control}, π — all scalar parameters. CIs span zero; point estimates are biased toward zero (prior mean) due to MAP shrinkage under near-flat likelihood.
- **Not separately reported**: brand fixed effects α_j (absorbed into DeepHalo embedding); individual discount factors δ_i (fixed at mean of true distribution per Magnac-Thesmar non-identification result); Halo network weights (weight-level comparison across different random initialisations is not meaningful).

Path to full scalar identification. Based on the Hessian diagnostic, recovering $\hat{\beta}_{\text{price}}$ with a CI entirely negative requires SE < 0.45, i.e. $|H_{\beta\beta}| > 4.9$ — approximately 13× the current curvature. Under standard $1/\sqrt{N}$ SE scaling this requires $N \approx 40{,}000$ observations, achievable at $I = 2{,}000$ consumers (holding $T = 20$). Additional instruments beyond w_{jt} would also increase curvature by providing independent price variation.

19.6 Coverage study at the correct DGP scale

A multi-seed coverage study is run at $T = 20$, $I = 150$, $n_{\text{seeds}} = 3$ — the same DGP scale as the simulation study — using `coverage_study.py`. Results are stored in `results/bonus1/coverage_study/coverage_study.json`.

Across three independent DGP draws, empirical 95% coverage is 100% for β_{price}, κ_0, μ_t, and d_{jt}. The mean $\hat{\beta}_{\text{price}}$ across seeds is -1.040 (true $= -1.5$; std $= 0.276$), confirming that the point-estimate bias is consistent and not a single-seed artefact. Mean $\hat{\kappa}_0 = 1.107$ (true $= 2.0$) and mean $\text{corr}(\hat{\mu}, \mu^\star) = 0.583$, consistent with the single-draw results. As explained above, 100% coverage reflects interval width rather than calibration: when SE $\gg |\hat{\theta} - \theta^\star|$, the CI trivially contains the truth. Meaningful coverage calibration requires either a much larger sample (so SEs narrow below the estimation bias) or a simplified model where scalar parameters are more sharply identified.

19.7 What the simulation demonstrates

The simulation study establishes four results:

1. The estimation pipeline is mechanically correct end-to-end: data generation, two-stage training, Petrin & Train control function, Laplace credible intervals, and counterfactual analysis all execute without numerical failure in graph mode with no retracing.
2. Consumer-specific tastes η_{ij} are identified from panel structure: RMSE $\approx \sigma_\eta^\star = 0.500$ confirms noise-floor recovery at $T = 20$ observations per consumer.
3. Market-level shocks μ_t are recovered in rank order (corr = 0.737 with CF) but with compressed magnitude — the expected MAP shrinkage pattern under weak likelihood signal.
4. Scalar parameters (β_{price}, κ_0, λ_{control}, π) are weakly identified at $N = 3{,}000$. The CF correction moves $\hat{\beta}$ from -0.779 (52% recovery) to -0.881 (59% recovery) with the correct positive sign on $\hat{\lambda}_{\text{control}}$. Full magnitude recovery requires $I \gtrsim 2{,}000$ or additional instruments.

20 Counterfactual Price Promotion Analysis

This section uses the fitted model to evaluate the effect of a price promotion on the expected revenue of a chosen brand X. The exercise is implemented in `jpm_q3.bonus1.dynamic_model.counterfactual.price_promotion_analysis`.

20.1 Formulation

Let $\hat{\theta}_{\text{est}}$ denote the fitted parameter vector from Section 18. Define the baseline expected *revenue per consumer-period* as

$$R_X^{\text{base}}(\hat{\theta}) \;=\; \frac{1}{N}\sum_{n=1}^{N} \hat{P}(X \mid s_n, \mathcal{A}_{t(n)}, p_{t(n)}; \hat{\theta})\, p_{X,t(n)}, \tag{79}$$

i.e. the expected payment to brand X across all observations.

A price promotion of x% off brand X is implemented by replacing the brand-X price column in every observation with $p_{X,t}^{\text{promo}} = (1 - x/100) \cdot p_{X,t}$ and leaving all other prices unchanged. The promoted revenue is

$$R_X^{\text{promo}}(\hat{\theta}) \;=\; \frac{1}{N}\sum_{n=1}^{N} \hat{P}(X \mid s_n, \mathcal{A}_{t(n)}, p_{t(n)}^{\text{promo}}; \hat{\theta})\, p_{X,t(n)}^{\text{promo}}. \tag{80}$$

The percentage revenue change is

$$\Delta R_X \;=\; 100 \cdot \frac{R_X^{\text{promo}} - R_X^{\text{base}}}{|R_X^{\text{base}}|}. \tag{81}$$

We additionally report brand X's expected market share before and after the promotion.

20.2 Numerical example

The demo run (`run_demo.py`) executes the counterfactual with brand $X = 1$ and discount $x = 10\%$. Selected outputs:

Quantity	Value
Revenue baseline	0.147
Revenue promotion	0.149
Revenue change	+0.001 (+0.86%)
Share X baseline	0.149
Share X promotion	0.161
Share X change	+0.0122

20.3 Mediation by Halo and stockpiling

The Halo effect mediates the promotion through context-dependent substitution. In this counterfactual the menu $\mathcal{A}_t$ is held fixed, so the price cut does not mechanically change the Halo term itself. Instead, Halo changes the diversion pattern: the softmax response to brand X's lower price depends on which other brands appear in the menu in that period. A promotion run when X shares the menu with close substitutes (captured by Halo) can therefore produce a different share gain than a promotion run when X is the unique premium option.

Stockpiling mediates the promotion through inventory dynamics. Consumers with high s_{it} have already absorbed the cost of past purchases and have a low marginal benefit from buying again at the promoted price — the inventory transition (63) caps them at $S_{\max}$. Consumers with low s_{it} have a strong stockout-avoidance motive (κ_0) and are more responsive to the promotion. The net revenue effect therefore depends on the joint distribution of inventory across consumers at the time the promotion is offered.

20.4 Interpretation of the positive revenue change

In the demo run, the promotion raises brand-X expected revenue by 0.86%. The fitted price coefficient remains attenuated relative to the DGP ($\hat{\beta}_{\text{price}} \approx -0.94$ vs. true -1.5), but the brand-specific share response from the 10% price cut (Δshare $= +0.0122$) is large enough to offset the lower unit price. This illustrates why the revised counterfactual reports revenue for the promoted brand only: category-wide expected revenue can move differently from brand-X revenue when substitution across inside goods is substantial.

20.5 Limitations

The counterfactual treats the price path as exogenously fixed under the promotion scenario. In a fully closed-form analysis, lowering the brand-X price would shift the unobserved shock $\xi_{X,t}$ if the promotion signals new demand-side information; we do not model that feedback. We also hold the choice set $\mathcal{A}_t$ fixed, which is relaxed in Part 5 where brand X jointly chooses price and assortment timing.

21 Joint Optimisation of Price Promotion and Assortment Timing

In this section, brand X chooses both a price promotion schedule and the days on which it appears in the choice set, subject to a cardinality constraint on appearance days. A full implementation is not required; we present a rigorous formulation and discuss an algorithm for finding the optimal policy.

21.1 Problem statement

Given the fitted demand model $\hat{P}(j \mid s, \mathcal{A}_t, p_t; \hat{\theta})$ from Section 18, brand X chooses:

- A *price discount schedule* $x = (x_1, \ldots, x_T) \in [0, x_{\max}]^T$, where x_t is the percentage discount applied at period t and $p_{X,t}^{\text{actual}} = (1 - x_t/100) \cdot p_{X,t}$.
- An *assortment indicator* $z = (z_1, \ldots, z_T) \in \{0, 1\}^T$, where $z_t = 1$ denotes that brand X appears in $\mathcal{A}_t$ and $z_t = 0$ denotes it is absent.

When $z_t = 0$ the choice set $\mathcal{A}_t$ is replaced by $\mathcal{A}_t \setminus \{X\}$, so the brand contributes no revenue at t regardless of x_t.

21.2 Objective

Let $R_X(x, z; \hat{\theta})$ denote brand X's expected total revenue across the planning horizon:

$$R_X(x, z; \hat{\theta}) \;=\; \sum_{t=1}^{T} z_t \;\cdot\; (1 - x_t/100)\, p_{X,t} \;\cdot\; \mathbb{E}_{i,\,s}\big[\hat{P}(X \mid s, \mathcal{A}_t^{(z)}, p_t^{(x)}; \hat{\theta})\big], \tag{82}$$

where the expectation is over the consumer index i and the inventory state s, which itself evolves stochastically under (x, z) through (63).

21.3 Constraint

Brand X may appear in the choice set for exactly K periods out of T:

$$\sum_{t=1}^{T} z_t \;=\; K, \qquad K \in \{1, \ldots, T\} \text{ fixed.} \tag{83}$$

21.4 Joint optimisation problem

$$\boxed{(x^\star, z^\star) \;=\; \arg\max_{x,\,z}\; R_X(x, z; \hat{\theta}) \quad \text{s.t.} \quad \sum_{t=1}^{T} z_t = K, \;\; z \in \{0, 1\}^T, \;\; x \in [0, x_{\max}]^T.} \tag{84}$$

21.5 Structure and challenges

The optimisation has three sources of difficulty.

Combinatorial assortment. The number of feasible z is $\binom{T}{K}$. For $T = 20, K = 10$ this is over $1.8 \cdot 10^5$ — too large for naive enumeration but small enough to admit branch-and-bound or relaxation methods.

Continuous price schedule. Conditional on z, the inner problem is to choose $x \in [0, x_{\max}]^T$ continuously. The MNL choice probability $\hat{P}(X \mid \cdot)$ is smooth in x_t, so the inner problem is amenable to gradient-based methods.

Dynamic coupling through inventory. A discount at period t shifts consumers' inventory at $t+1$, which in turn affects the optimal discount at $t+1$. The objective is therefore *not* separable across periods even conditional on z. This is the standard intertemporal coupling problem from dynamic stockpiling.

21.6 Proposed algorithm

We propose a nested approach that separates the discrete and continuous decisions.

Algorithm 2 Joint optimisation of price and assortment timing

Require: Fitted model $\hat{\theta}$, horizon T, cardinality K, max discount $x_{\max}$

Outer loop (assortment, discrete):
Initialise $z^{(0)}$ by greedy selection (see below)
for iter $= 1, \ldots, M$ **do**
    Inner loop (price, continuous):
    for Adam steps $k = 1, \ldots, K_{\text{inner}}$ **do**
        Compute $\nabla_x R_X(x, z^{(\text{iter}-1)}; \hat{\theta})$ via automatic differentiation through the model
        Project step onto $[0, x_{\max}]^T$ (clip)
    end for
    Let $x^\star(z^{(\text{iter}-1)})$ be the inner solution
    Swap move: pick the worst-contribution period $t \in \text{supp}(z)$ and the best-counterfactual period $t' \notin \text{supp}(z)$; swap $z_t \leftrightarrow z_{t'}$ if revenue improves
    Accept the swap into $z^{(\text{iter})}$
end for
return $(x^\star, z^\star)$

Outer (discrete) greedy initialisation. Initialise $z^{(0)}$ by sorting periods by their *stand-alone revenue potential* — the expected revenue when brand X is the only discounted brand and inventory is sampled from its steady-state distribution — then setting $z_t = 1$ for the top-K periods. This is a standard greedy heuristic and provides a strong warm start.

Inner (continuous) gradient descent. Conditional on z, the inner problem is a smooth box-constrained optimisation that can be solved by projected gradient descent (Adam) with the gradient computed end-to-end through the fitted model. Each Adam step costs one forward pass of the dynamic choice model.

Outer (discrete) refinement via swaps. After the inner solve, refine z via 1-swap moves: identify the in-support period with the smallest revenue contribution and the out-of-support period with the largest counterfactual contribution, and swap them if it improves the objective. Iterate until no 1-swap improves.

21.7 Connections to related approaches

Lagrangian relaxation. An alternative to the inner-outer scheme is to relax the cardinality constraint into a Lagrangian

$$\mathcal{L}_\lambda(x, z, \lambda) \;=\; R_X(x, z; \hat{\theta}) \;-\; \lambda\Big(\sum_t z_t - K\Big), \tag{85}$$

treat $z_t \in [0, 1]$ as a continuous relaxation, optimise (x, z, λ) jointly, and round z at the end. This loses optimality guarantees but is fully gradient-based.

Dynamic programming with reduced state. A different approach is to treat the problem as a finite-horizon dynamic program over $(s_t, t, \text{remaining assortment slots})$. The state space is $O(S_{\max} \cdot T \cdot K)$, which is tractable for the question's dimensions but requires per-state value computation.

Constrained policy search. For larger settings, the problem is amenable to constrained policy search via REINFORCE with a Lagrangian penalty (constrained policy gradient). This is the right approach if the horizon or cardinality is too large for branch-and-bound.

21.8 Suitability for the credit-card offer setting

In the credit-card setting that motivates this project, brand X is a merchant offer. The cardinality constraint $\sum z_t = K$ corresponds to a fixed promotional budget over the campaign window. The price discount x_t corresponds to the cashback percentage on a given day. The proposed algorithm directly fits this operational setting: outer

greedy + swap selects which days to run the offer, inner gradient descent tunes the discount on those days, all conditional on the demand model fitted from historical redemption data.

The Halo effect is particularly important here because cashback offers typically appear alongside competitor merchant offers in a curated menu. Adding a competitor to the menu on day t shifts both the substitution pattern and the counterfactual revenue, and the DeepHalo network captures this menu-level effect in a way that linear demand models cannot.

22 Known Limitations

The model and results presented in this part are a research prototype implemented and validated on synthetic data. The following limitations are explicitly acknowledged.

1. **Synthetic-only validation.** All models are validated on DGPs generated by the code in this repository. No real credit-card transaction data were used. Parameter-recovery results (Section 19) speak to the estimator's behaviour on a known DGP, not to its performance on real data where the true parameter values are unknown.

2. **Approximate MCMC (Part 2 shrinkage).** The TFP `RandomWalkMetropolis` implementation deviates from the collapsed Gibbs sampler that would be available in a purely conjugate spike-and-slab model. Chain lengths ($n_{\text{iter}} = 800$, $n_{\text{burn}} = 400$) are chosen for computational tractability; production-quality inference would require ≥ 5000 post-burn-in draws and convergence diagnostics ($\hat{R}$, effective sample size).

3. **Weak identification of β_{price} at this scale.** The 95% Laplace credible interval for β_{price} is $[-4.40, +2.22]$ on the reported run, which crosses zero and is too wide to be practically informative. This reflects the combined challenges of a small panel ($T = 20$, $I = 150$), dynamic confounding between the value-head embedding and the price coefficient, and a single excluded instrument. A panel with $T \geq 50$ or $I \geq 500$ and multiple instruments would substantially narrow this interval.

4. **Fixed discount factor.** Consumer-specific discount factors δ_i are not estimated. A common $\delta = 0.825$ (the mean of the true Uniform$(0.70, 0.95)$ distribution) is used throughout estimation. Magnac & Thesmar (2002) show that discount factors are not point-identified without additional restrictions in single-agent dynamic discrete choice models; the fixed-δ approximation is standard practice in applied work.

5. **No proof of global identifiability.** The model is identified by a combination of IV exclusion restrictions (observable cost shifter w_{jt} excluded from utility), sparsity of d_{jt}, and normalisation choices (within-market centering of d, outside option utility normalised to zero for inside goods). No formal identification theorem is proved. In particular, joint identification of the Halo weights, μ_t, d_{jt}, and η_{ij} under the estimated model has not been established.

6. **Computational scaling of Laplace intervals.** The exact Hessian diagonal computation in `compute_laplace_intervals` uses `tf.jacobian` with $O(n^2)$ complexity in the parameter count, where $n = T \cdot J$ for the d matrix. For the default configuration ($T = 20$, $J = 5$) this is $n = 100$, which is tractable. For $T \times J > 500$ the computation becomes prohibitively expensive and a diagonal Fisher approximation or sub-sampled Hessian would be needed.

7. **TF 2.16/Keras 3 embedding-gradient limitation.** As detailed in Section 18, integer-indexed Keras embedding lookups do not receive gradients through `@tf.function` in this TF version. The impact on econometric parameters is minimal for the fixed-universe setting used here but would be material in a setting where item identities vary across observations.

References

[1] S. Zhang, Z. Wang, R. Gao, and S. Li, "Deep context-dependent choice model," in *2nd Workshop on Models of Human Feedback for AI Alignment*, 2025. [Online]. Available: https://openreview.net/forum?id=bXTBtUjb0c

[2] Z. Lu and K. Shimizu, *Estimating discrete choice demand models with sparse market-product shocks*, 2025. arXiv: 2501.02381 [econ.EM]. [Online]. Available: https://arxiv.org/abs/2501.02381

[3] Y. Yang, Z. Wang, R. Gao, and S. Li, "Reproducing kernel hilbert space choice model," in *Proceedings of the 26th ACM Conference on Economics and Computation*, ser. EC '25, Stanford University, Stanford, CA, USA: Association for Computing Machinery, 2025, p. 819, ISBN: 9798400719431. DOI: 10.1145/3736252.3742630 [Online]. Available: https://doi.org/10.1145/3736252.3742630

[4] A. Petrin and K. Train, "A control function approach to endogeneity in consumer choice models," *Journal of Marketing Research*, vol. 47, no. 1, pp. 3–13, 2010. DOI: 10.1509/jmkr.47.1.3

[5] B. Efron, *Large-Scale Inference: Empirical Bayes Methods for Estimation, Testing, and Prediction* (Institute of Mathematical Statistics Monographs). Cambridge University Press, 2012, ISBN: 9781107619678.